

Introduzione alla Semantica

G. Michele Pinna

24 novembre 2021

Dare la semantica ad un programma serve a definire con esattezza *cosa* fa questo programma, e quindi anche a sviluppare tecniche di ottimizzazione, di debugging, oppure fare verifiche di correttezza, fare traduzione fra formalismi diversi, e molto altro. Nel nostro corso servirà principalmente a scrivere un interprete per questo linguaggio e capire cosa serve per realizzare la *macchina astratta*, che definiremo un po' più formalmente in seguito.

La definizione semi-formale della semantica fa parte dello standard di definizione di un linguaggio di programmazione a partire dagli anni '80, ma la descrizione di come devono funzionare i costrutti ha bisogno di un approccio formale.

Prima di discutere più formalmente cosa è la *semantica* di un linguaggio di programmazione, e come questa si dà, diciamo com'è un *paradigma*.

Paradigma di programmazione

Un *paradigma* di programmazione è uno un insieme di strumenti concettuali forniti da un linguaggio di programmazione per la stesura del codice sorgente di un programma, e definisce il modo in cui il programmatore concepisce e percepisce il programma stesso, e quindi anche la sua esecuzione.

I diversi paradigmi si differenziano per i concetti e le astrazioni usate per rappresentare gli elementi di un programma, come ad esempio le funzioni, gli oggetti, le variabili o i vincoli, e anche per i procedimenti usati per l'esecuzione delle procedure di elaborazione dei dati, come l'assegnamento, l'iterazione, la ricorsione, il data flow, l'applicazione di funzioni.

Facciamo un elenco di paradigmi di programmazione, senza la pretesa di esaurirli tutti.

- **IMPERATIVO:** Nel paradigma imperativo è il programmatore a stabilire quale sarà l'istruzione successiva. Quindi è la scrittura del programma stesso a dire quale sarà la sua esecuzione. Il modello di base è quello dei primi linguaggi di programmazione, come il FORTRAN, quindi spesso ci si concentra sull'algoritmo visto come sequenza di passi. Un passo di computazione in generale modifica lo *stato* (concetto che vedremo meglio dopo).

- **FUNZIONALE:** in questo paradigma il programmatore definisce quali sono le funzioni e come queste vengono invocate ed usate. Il focus non è tanto sull'algoritmo come sequenza di passi ma su quali sono le funzioni per risolvere il problema e quali risultati producono. Il modello è il λ -calcolo. Un passo di computazione in generale restituisce un valore.
- **PROCEDURALE:** In questo paradigma si prova a coniugare la parte imperativa con la definizione di funzioni, che in questo paradigma vengono chiamate procedure. Un passo di computazioni in generale ha un effetto sullo stato ma anche, potenzialmente, restituisce un valore. I linguaggi come PASCAL o ALGOL possono essere presi come linguaggi propri di questo paradigma.
- **OBJECT ORIENTED:** In questo paradigma le varie parti di programma interagiscono scambiandosi messaggi. Sono quindi questi messaggi, scambiato tra parti di programma che sono visti come oggetti medesimi. Il linguaggio di programmazione paradigmatico è lo SMALLTALK, e la sua precisa semantica è quella algebrica.
- **LOGICO:** la logica, in particolare le clausole di Horn, sono alla base di questo paradigma di programmazione. Il linguaggio proprio di questo paradigma è il PROLOG.

Metodi per dare la semantica

Ci sono vari metodi per definire la semantica di un linguaggio di programmazione, che qui elenchiamo, cercando di evidenziarne le caratteristiche salienti, senza alcuna pretesa di completezza.

- **Algebrica.** Nasce agli inizi anni degli 70, si parte dai tipi di dato che formano un'*algebra* (spesso detta multi-sorted). La sintassi è data dalla cosiddetta *term algebra*, le operazioni nelle algebre semantiche sono definite tramite *equazioni*, e la semantica è l'omeomorfismo più generale dalla term algebra alle algebre semantiche. Concetti come tipo di dato astratto (moduli, classi, etc) sono ben descritti da questa semantica.
- **Assiomatica.** Tra le prime ad essere formalizzata, caratterizza gli *stati* (proprietà), e per ogni costrutto si cerca di capire come è la *trasformazione* indotta nello stato formalizzata come proprietà dello stato e si basa su una descrizione (più o meno formale) di cosa fa un costrutto. Tale semantica è molto indicata per la verifica di *correttezza* di un programma, e quindi per lo sviluppo di programmi corretti.
- **Operazionale.** Ottenuta definendo un interprete del linguaggio su di una macchina ospite le cui componenti sono descritte in modo matematico. Lo *stato* descrive complessivamente il sistema, e la *transizione* specifica un passo di computazione che modifica lo stato e/o il programma. Una computazione è una sequenza di transizioni. È utile soprattutto perché fornisce direttamente un modello di implementazione

- Denotazionale. Formalizza quale è la trasformazione sugli stati indotta dal programma, sostanzialmente una funzione dagli input agli output di un programma, ed è focalizzata su quale è la trasformazione: *denoto* = esprimo con chiarezza, indico e non su come tale trasformazione avviene, e poggia su basi matematiche sostanzialmente sviluppate negli anni 70.

Noi ci focalizzeremo sulla semantica operativa, e quindi precisiamo maggiormente quanto detto prima.

Semantica Operativa

Possiamo definirla in modo formale, guidati dalla sintassi di un linguaggio di programmazione, o meglio dalla sintassi dei vari costrutti. Seguiamo il cosiddetto approccio SOS (Structured Operational Semantics). Per prima cosa dobbiamo definire cosa è lo *stato* rispetto al quale *valutiamo* (eseguiamo) un costrutto.

La semantica operativa è per noi definita rispetto a una nozione di stato. Dato un programma p , con $\text{Name}(p)$ indichiamo l'insieme dei *nomi* che compaiono nel programma. Saranno quindi tutti gli *identificatori* come i nomi di variabili o simili. Non ne diamo una definizione formale. I valori che possono essere associati ai vari nomi li indichiamo con *Value* e non dipendono dal programma medesimo, e anche in questo caso non ne diamo una definizione formale.

Definizione 1 *Dato un programma p , lo stato è un'associazione tra $\text{Name}(p)$ e Value.*

La definizione è generica, dato che non si specifica precisamente come è fatta l'associazione tra nomi e valori. Quindi, per noi (e per ora), lo stato può essere una sequenza finita di coppie (in cui il primo elemento, cioè il nome della variabile, compare una sola volta). Ogni coppia (X, n) ha come primo elemento il nome (in questo caso X) e come secondo elemento il valore (in questo caso n). Possiamo più semplicemente vedere lo stato come una funzione dai nomi ($\text{Name}(p)$) ai valori associabili ai nomi (*Value*).

La nozione di stato dipende anche molto dal paradigma che consideriamo. Ad esempio, nel paradigma imperativo e in quello procedurale, lo stato ha una parte che chiameremo ambiente (una funzione da nomi ad oggetti che chiameremo *denotabili*), una memoria, che sarà una funzione da locazioni (che assumeremo denotabili) a oggetti memorizzabili, uno heap, che sarà una struttura dati per la memorizzazione delle strutture dati dinamiche, mentre nel paradigma funzionale avremo solo l'ambiente, cioè l'associazione tra nomi e oggetti denotabili.

Prima di introdurre un qualche linguaggio e di definirne la semantica, bisogna stabilire cosa fa una semantica operativa. Ovviamente questo dipende dal linguaggio e dal paradigma a cui tale linguaggio appartiene. Nel caso di linguaggi imperativi la semantica operativa definisce un cambiamento di *stato*, mentre nei linguaggi funzionali la semantica operativa restituisce il risultato della valutazione, quindi un *valore*.

Una *computazione* è in generale una catena di cambiamenti, di stato nel caso imperativo e di valori nel caso funzionale. Questi cambiamenti vengono descritti da opportune

regole, che hanno la forma:

$$\frac{\text{premesse}}{\text{conseguenza}}$$

ed in generale le regole sono guidate dalla sintassi. In generale le transizioni per un costruito composto $A \text{ op } B$ sono date in termini di quelli dei componenti A e B , che saranno esplicitate nelle premesse. Se non ci sono premesse allora la regola è un assioma.

Il linguaggio IMP

Vediamo il nucleo di un linguaggio *imperativo*, che chiamiamo IMP. Il linguaggio IMP che viene considerato ha un insieme di identificatori (che prima abbiamo chiamato **Name**) e che ora chiamiamo *Var* mentre i valori in **Value** saranno i numeri interi. Lo stato lo vediamo come una funzione da *Var* a $\mathbb{Z} \cup \{\perp\}$ dove $\perp$ indica che ad un certo identificatore non è associato alcun valore. Quindi l'insieme degli stati è

$$\Sigma = \{\sigma \mid \sigma : \text{Var} \rightarrow \mathbb{Z} \cup \{\perp\}\}$$

Per dire che, dato un nome X ed uno stato σ , al nome X è associato un valore, scriviamo $\sigma(X) \downarrow$.

Espressioni aritmetiche. Diamo prima le regole per la valutazione delle espressioni aritmetiche di un linguaggio imperativo. La sintassi di queste espressioni è

$$a ::= n \mid X \mid a_0 + a_1 \mid a_0 \times a_1 \mid a_0 - a_1$$

dove $X \in \text{Var}$ sono le *variabili*.

Per valutare un'espressione abbiamo bisogno dello stato, che abbiamo detto indichiamo con σ . Le regole sono:

$$\begin{array}{c} \frac{}{\langle n, \sigma \rangle \longrightarrow n} \qquad \frac{\sigma(X) \downarrow}{\langle X, \sigma \rangle \longrightarrow \sigma(X)} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 + a_1, \sigma \rangle \longrightarrow n = n_0 + n_1} \\[10pt] \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \times a_1, \sigma \rangle \longrightarrow n = n_0 \times n_1} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 - a_1, \sigma \rangle \longrightarrow n = n_0 - n_1} \end{array}$$

Vediamo un esempio di come valutare un'espressione. Prendiamo un'espressione: $4 + 5 - 2$. Lo stato in questo caso non influenza la valutazione dato che nell'espressione non appaiono variabili. Le regole usano metavariables che sono $n, a_0, \dots$ e che hanno valori sui vari domini sintattici, e una istanza di una regola si ottiene istanziando queste metavariables a particolari numeri, variabili, espressioni e simili. Per valutare $4 + 5 - 2$ applichiamo prima la regola

$$\frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 + a_1, \sigma \rangle \longrightarrow n = n_0 + n_1}$$

istanziata in questo modo:

$$\frac{\langle 4, \sigma \rangle \longrightarrow n_0 \quad \langle 5 - 2, \sigma \rangle \longrightarrow n_1}{\langle 4 + 5 - 2, \sigma \rangle \longrightarrow n = n_0 + n_1}$$

Le metavariables sono a_0 ed a_1 , e a_0 è 4, a_1 è $5 - 2$, le altre per ora le lasciamo non specificate.

Sappiamo che dobbiamo valutare ora 4 e $5 - 2$. Valutiamo 4. La regola è

$$\overline{\langle n, \sigma \rangle \longrightarrow n}$$

che istanziata diventa

$$\overline{\langle 4, \sigma \rangle \longrightarrow 4}$$

Valutiamo $5 - 2$, la regola da istanziare è ovvia, e la sua istanziazione è

$$\frac{\langle 5, \sigma \rangle \longrightarrow n_0 \quad \langle 2, \sigma \rangle \longrightarrow n_1}{\langle 5 - 2, \sigma \rangle \longrightarrow n = n_0 - n_1}$$

Valutiamo 5 e 2. Abbiamo

$$\overline{\langle 5, \sigma \rangle \longrightarrow 5} \quad \text{e} \quad \overline{\langle 2, \sigma \rangle \longrightarrow 2}$$

ottenendo

$$\frac{\langle 5, \sigma \rangle \longrightarrow 5 \quad \langle 2, \sigma \rangle \longrightarrow 2}{\langle 5 - 2, \sigma \rangle \longrightarrow 3 = 5 - 2}$$

ed infine

$$\frac{\langle 4, \sigma \rangle \longrightarrow 4 \quad \langle 5 - 2, \sigma \rangle \longrightarrow 3}{\langle 4 + 5 - 2, \sigma \rangle \longrightarrow 7 = 4 + 3}$$

Facciamo alcune considerazioni, che però non hanno a che fare con le regole della semantica. La grammatica che abbiamo visto ha il problema dell'*ambiguità* (dobbiamo ricorrere alla precedenza degli operatori per valutare correttamente l'espressione), prendiamone una che ha le stesse derivazioni (l'idea sarà di dare la grammatica che genera l'albero di derivazione). La sintassi (astratta) è

$$a ::= n \mid val(X) \mid piu(a_0, a_1) \mid per(a_0, a_1) \mid meno(a_0, a_1)$$

Le regole non cambiano.

$$\begin{array}{ccc} \overline{\langle n, \sigma \rangle \longrightarrow n} & \frac{\sigma(X) \downarrow}{\langle val(X), \sigma \rangle \longrightarrow \sigma(X)} & \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle piu(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 + n_1} \\ \\ \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle per(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 \times n_1} & & \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle meno(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 - n_1} \end{array}$$

Quando abbiamo valutato l'espressione abbiamo costruito una derivazione, le istanze delle regole si compongono in strutture ad albero dette alberi di derivazione dove:

- tutte le premesse di istanze di regole sono conclusioni di istanze di regole immediatamente sopra nell'albero;
- Le estremità (foglie) dell'albero sono assiomi (regole senza premesse)

Estendiamo il frammento. Abbiamo espressioni aritmetiche e booleane la cui sintassi è

$$\begin{aligned}
 a & ::= n \mid X \mid \text{piu}(a_0, a_1) \mid \text{per}(a_0, a_1) \mid \text{meno}(a_0, a_1) \\
 b & ::= \text{true} \mid \text{false} \mid \text{or}(b_0, b_1) \mid \text{and}(b_0, b_1) \mid \text{not}(b_0) \\
 & \quad \mid \text{uguale}(a_0, a_1) \mid \text{minore}(a_0, a_1)
 \end{aligned}$$

Le regole per le espressioni aritmetiche le abbiamo sostanzialmente già viste:

$$\begin{array}{c}
 \frac{}{\langle n, \sigma \rangle \longrightarrow n} \qquad \frac{\sigma(X) \downarrow}{\langle X, \sigma \rangle \longrightarrow \sigma(X)} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{piu}(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 + n_1} \\
 \\
 \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{per}(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 \times n_1} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{meno}(a_0, a_1), \sigma \rangle \longrightarrow n = n_0 - n_1}
 \end{array}$$

vediamo anche quelle delle espressioni booleane. Qui dobbiamo dire quali sono i valori che la valutazione di una espressione booleana restituisce, e questi sono $\mathbb{B} = \{\text{true}, \text{false}\}$. Questi fanno in qualche modo parte dei Value che ora sono $\mathbb{Z} \cup \mathbb{B}$.

$$\begin{array}{c}
 \frac{}{\langle \text{true}, \sigma \rangle \rightsquigarrow \text{true}} \qquad \frac{}{\langle \text{false}, \sigma \rangle \rightsquigarrow \text{false}} \qquad \frac{\langle b, \sigma \rangle \rightsquigarrow t}{\langle \text{not}(b), \sigma \rangle \rightsquigarrow \neg t} \\
 \\
 \frac{\langle b_0, \sigma \rangle \rightsquigarrow t_0 \quad \langle b_1, \sigma \rangle \rightsquigarrow t_1}{\langle \text{or}(b_0, b_1), \sigma \rangle \rightsquigarrow t_0 \vee t_1} \qquad \frac{\langle b_0, \sigma \rangle \rightsquigarrow t_0 \quad \langle b_1, \sigma \rangle \rightsquigarrow t_1}{\langle \text{and}(b_0, b_1), \sigma \rangle \rightsquigarrow t_0 \wedge t_1} \\
 \\
 \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{uguale}(a_0, a_1), \sigma \rangle \rightsquigarrow n_0 = n_1} \qquad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle \text{minore}(a_0, a_1), \sigma \rangle \rightsquigarrow n_0 < n_1}
 \end{array}$$

Comandi Vediamo ora la sintassi dei comandi di un semplice linguaggio imperativo:

$$\begin{aligned}
 c & ::= \text{skip} \\
 & \quad \mid \text{assign}(X, a) \\
 & \quad \mid \text{cseq}(c_0, c_1) \\
 & \quad \mid \text{ifthenelse}(b, c_0, c_1) \\
 & \quad \mid \text{while}(b, c)
 \end{aligned}$$

i comandi usano anche le variabili, le espressioni aritmetiche e quelle booleane.

Per poterne dare la semantica dobbiamo prima capire cosa fa un comando in generale. La risposta, come in questi casi capita sempre, è ovvia: un comando modifica lo stato, e nel nostro caso lo stato non sarà altro che un'associazione tra i nomi, che qui sono le variabili, ed i valori associabili alle variabili, che qui sono gli interi.

Lo stato lo aggiorniamo quando assegniamo un valore ad una variabile. La notazione è la seguente: $\sigma[n/X]$ che significa che abbiamo assegnato n alla variabile X . $\sigma[n/X]$ è la funzione dalle variabili Var ai valori $\mathbb{Z}$ che

$$\sigma[n/X](Y) = \begin{cases} n & \text{se } X = Y \\ \sigma(Y) & \text{altrimenti} \end{cases}$$

Vediamo ora la semantica dai comandi.

- **skip**. Questo comando non fa nulla, lascia invariato lo stato. La sua semantica può essere data con la regola

$$\frac{}{\langle \text{skip}, \sigma \rangle \Longrightarrow \sigma}$$

- L'assegnamento: il comando **assign**(X, a) modifica lo stato assegnando alla variabile X il risultato della valutazione dell'espressione aritmetica a .

$$\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle \text{assign}(X, a), \sigma \rangle \Longrightarrow \sigma[n/X]}$$

- il ';', cioè quello che stabilisce la sequenza di due comandi. La regola è ovvia: devo eseguire il secondo nello stato ottenuto eseguendo il primo

$$\frac{\langle c_0, \sigma \rangle \Longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \Longrightarrow \sigma'}{\langle \text{cseq}(c_0, c_1), \sigma \rangle \Longrightarrow \sigma'}$$

- il condizionale (If then else). In questo caso ho bisogno di due regole, dato che devo capire, in base al valore dell'espressione booleana, quale dei due comandi eseguire.

$$\frac{\langle b, \sigma \rangle \mapsto \text{true} \quad \langle c_0, \sigma \rangle \Longrightarrow \sigma'}{\langle \text{ifthenelse}(b, c_0, c_1), \sigma \rangle \Longrightarrow \sigma'}$$

e

$$\frac{\langle b, \sigma \rangle \mapsto \text{false} \quad \langle c_1, \sigma \rangle \Longrightarrow \sigma'}{\langle \text{ifthenelse}(b, c_0, c_1), \sigma \rangle \Longrightarrow \sigma'}$$

- l'iterazione (il while). Anche in questo caso ho due regole. Se la condizione booleana è falsa, allora non devo fare nulla, se invece è vera devo prima eseguire il comando c e, nello stato ottenuto, eseguire il costrutto di iterazione, cioè il while.

$$\frac{\langle b, \sigma \rangle \mapsto false}{\langle \mathbf{while}(b, c), \sigma \rangle \Longrightarrow \sigma}$$

e

$$\frac{\langle b, \sigma \rangle \mapsto true \quad \langle c, \sigma \rangle \Longrightarrow \sigma'' \quad \langle \mathbf{while}(b, c), \sigma'' \rangle \Longrightarrow \sigma'}{\langle \mathbf{while}(b, c), \sigma \rangle \Longrightarrow \sigma'}$$

Le regole della semantica operativa ci sono servite per definire tre relazioni:

- $\longrightarrow \subseteq Aexp \times \Sigma \times \mathbb{Z}$
- $\mapsto \subseteq Bexp \times \Sigma \times \mathbb{B}$
- $\Longrightarrow \subseteq Com \times \Sigma \times \Sigma$

dove $Aexp$ sono le espressioni aritmetiche, $Bexp$ le espressioni booleane e Com sono le istruzioni del nostro linguaggio IMP.

Queste relazioni sono *deterministiche* nel senso che, fissato un elemento in $Aexp$, $Bexp$ o Com ed uno stato, ogni relazione contiene una sola tripla che ha esattamente quei due primi elementi, quindi sostanzialmente definiscono delle funzioni. In particolare la relazione $\longrightarrow$ definisce la funzione $\mathcal{A} : Aexp \times \Sigma \rightarrow \mathbb{Z}$, la relazione $\mapsto$ definisce la funzione $\mathcal{B} : Bexp \times \Sigma \rightarrow \mathbb{B}$, ed infine la relazione $\Longrightarrow$ la funzione $\mathcal{C} : Com \times \Sigma \rightarrow \Sigma$.

La semantica ci aiuta anche a dire quando due costrutti sono *equivalenti*. Dato che a noi interessano nel linguaggio imperativo i comandi, formalizziamo cosa significa che due comandi sono equivalenti.

Definizione 2 Due comandi $c, c' \in Com$ sono equivalenti se e solo se per ogni stato $\sigma \in \Sigma$ vale che $\mathcal{C}(c, \sigma) = \mathcal{C}(c', \sigma)$.

Va osservato che la definizione di equivalenza può essere usata in due modi. Se una parte della semantica la definizione serve a caratterizzare quali sono i costrutti tra di loro equivalenti, se si parte invece dalla *richiesta* che certi costrutti siano equivalenti allora possiamo costruire la semantica in modo che tale equivalenza venga rispettata.

In questa semantica operativa abbiamo posto l'accento sulla trasformazione di stato, immaginando che tutti i comandi vengano eseguiti *integralmente*, dove integralmente significa che l'esecuzione del comando restituisce uno stato e non una coppia (costrutto, stato), prima di passare allo stato successivo. Potremmo però porre l'attenzione anche su cosa dobbiamo ancora eseguire. Di seguito le regole modificate tenendo conto di voler sottolineare, nella semantica, proprio questo.

$$\begin{array}{c}
\frac{}{\langle \mathbf{skip}, \sigma \rangle \Longrightarrow \sigma} \qquad \frac{\langle a, \sigma \rangle \longrightarrow n}{\langle \mathbf{assign}(X, n), \sigma \rangle \Longrightarrow \sigma[n/X]} \\
\\
\frac{\langle c_0, \sigma \rangle \Longrightarrow \sigma'}{\langle \mathbf{cseq}(c_0, c_1), \sigma \rangle \rightsquigarrow \langle c_1, \sigma' \rangle} \qquad \frac{\langle c_0, \sigma \rangle \rightsquigarrow \langle c'_0, \sigma' \rangle}{\langle \mathbf{cseq}(c_0, c_1), \sigma \rangle \rightsquigarrow \langle \mathbf{cseq}(c'_0, c_1), \sigma' \rangle} \\
\\
\frac{\langle b, \sigma \rangle \rightsquigarrow \mathbf{true}}{\langle \mathbf{ifthenelse}(b, c_0, c_1), \sigma \rangle \rightsquigarrow \langle c_0, \sigma \rangle} \qquad \frac{\langle b, \sigma \rangle \rightsquigarrow \mathbf{false}}{\langle \mathbf{ifthenelse}(b, c_0, c_1), \sigma \rangle \rightsquigarrow \langle c_1, \sigma \rangle} \\
\\
\frac{}{\langle \mathbf{while}(b, c), \sigma \rangle \rightsquigarrow \langle \mathbf{ifthenelse}(b, \mathbf{cseq}(c, \mathbf{while}(b, c)), \mathbf{skip}), \sigma \rangle}
\end{array}$$

La relazione che a noi interessa è sempre $\Longrightarrow \subseteq Com \times \Sigma \times \Sigma$ mentre la relazione $\rightsquigarrow \subseteq Com \times \Sigma \times Com \times \Sigma$ è una relazione *ausiliaria* che serve solo per definire $\Longrightarrow$.

Perché per noi è importante la semantica (operazionale)? Ha il vantaggio che rende molto semplice e naturale definire il nostro interprete: prendiamo le regole, che vediamo come delle funzioni, che scriviamo nel linguaggio di programmazione della macchina ospite. In più, è facile da usare, e ci serve per capire cosa dobbiamo mettere nel supporto a tempo d'esecuzione, cioè gli algoritmi e le strutture dati per l'esecuzione dei programmi del nostro linguaggio.

Il linguaggio REPEAT

Vediamo ora un linguaggio imperativo con costrutti leggermente diversi. Le espressioni aritmetiche e booleane non cambiano.

Vediamo per prima cosa la sintassi dei comandi:

$$\begin{array}{l}
c ::= \mathbf{skip} \mid \mathbf{break} \mid \mathbf{assign}(X, a) \mid \mathbf{cseq}(c_1, c_2) \mid \mathbf{ifthenelse}(b, c_1, c_2) \mid \\
\mathbf{repeat}(c)
\end{array}$$

il comando **break** interrompe la computazione. Viene usato sostanzialmente per interrompere la computazione di un **repeat** che *ripete* l'esecuzione del comando c indefinitamente. Gli altri costrutti sono standard.

Dobbiamo ora tenere conto che serve sapere se una computazione è stata interrotta o no. Per questo la cosa più naturale da fare è *modificare* la nozione di stato, che non è più solo una funzione dai nomi ai valori, ma è una coppia (σ, f) con $f \in \{\mathbf{ok}, \mathbf{br}\}$, dove $\sigma \in \Sigma$ e f è un flag che sostanzialmente dice se è stato eseguito un **break** o no.

Le regole sono le seguenti, ed in queste regole $\gamma \in \{\mathbf{ok}, \mathbf{br}\}$:

$$\begin{array}{c}
\frac{}{\langle \mathbf{skip}, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma, \mathbf{ok})} \qquad \frac{}{\langle \mathbf{break}, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma, \mathbf{br})} \\
\\
\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle \mathbf{assign}(X, a), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma[n/X], \mathbf{ok})} \qquad \frac{\langle c_1, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{br})}{\langle \mathbf{cseq}(c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{br})} \\
\\
\frac{\langle c_1, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma'', \mathbf{ok}) \quad \langle c_2, (\sigma'', \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)}{\langle \mathbf{cseq}(c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)} \\
\\
\frac{\langle b, \sigma \rangle \mapsto \mathbf{true} \quad \langle c_1, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)}{\langle \mathbf{ifthenelse}(b, c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)} \qquad \frac{\langle b, \sigma \rangle \mapsto \mathbf{false} \quad \langle c_2, (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)}{\langle \mathbf{ifthenelse}(b, c_1, c_2), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \gamma)} \\
\\
\frac{\langle c, (\sigma, \mathbf{ok}) \rangle \mapsto (\sigma', \mathbf{br})}{\langle \mathbf{repeat}(c), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{ok})} \\
\\
\frac{\langle c, (\sigma, \mathbf{ok}) \rangle \mapsto (\sigma'', \mathbf{ok}) \quad \langle \mathbf{repeat}(c), (\sigma'', \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{ok})}{\langle \mathbf{repeat}(c), (\sigma, \mathbf{ok}) \rangle \mapsto_c (\sigma', \mathbf{ok})}
\end{array}$$

Abbiamo introdotto un flag per rappresentare la condizione di *terminazione* di un'istruzione del linguaggio, aggiungendo questo flag allo stato. Avremmo però potuto fare diversamente. Avremmo potuto arricchire lo stato in questo modo. Consideriamo un nome che non appare in *Var*, ad esempio *flag*, possiamo modificare la nozione di stato in modo che $\sigma(X) \in \mathbb{Z} \cup \{\perp\}$ and $\sigma(\mathbf{flag}) \in \{\mathbf{ok}, \mathbf{br}\}$. Usando questa idea possiamo modificare la semantica che abbiamo dato in questo modo:

$$\begin{array}{c}
\frac{\sigma(\text{flag}) = \text{ok}}{\langle \text{skip}, \sigma \rangle \mapsto_c \sigma} \qquad \frac{\sigma(\text{flag}) = \text{ok}}{\langle \text{break}, \sigma \rangle \mapsto_c \sigma[\text{br}/\text{flag}]} \qquad \frac{\sigma(\text{flag}) = \text{ok} \quad \langle a, \sigma \rangle \longrightarrow n}{\langle \text{assign}(X, a), \sigma \rangle \mapsto_c \sigma[n/X]} \\
\\
\frac{\sigma(\text{flag}) = \text{ok} \quad \langle c_1, \sigma \rangle \mapsto_c \sigma' \quad \sigma'(\text{flag}) = \text{br}}{\langle \text{cseq}(c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\sigma(\text{flag}) = \text{ok} \quad \langle c_1, \sigma \rangle \mapsto_c \sigma'' \quad \sigma''(\text{flag}) = \text{ok} \quad \langle c_2, \sigma'' \rangle \mapsto_c \sigma'}{\langle \text{cseq}(c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\langle b, \sigma \rangle \mapsto \text{true} \quad \sigma(\text{flag}) = \text{ok} \quad \langle c_1, \sigma \rangle \mapsto_c \sigma'}{\langle \text{ifthenelse}(b, c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\langle b, \sigma \rangle \mapsto \text{false} \quad \sigma(\text{flag}) = \text{ok} \quad \langle c_2, \sigma \rangle \mapsto_c \sigma'}{\langle \text{ifthenelse}(b, c_1, c_2), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\sigma(\text{flag}) = \text{ok} \quad \langle c, \sigma \rangle \mapsto \sigma'' \quad \sigma''(\text{flag}) = \text{ok} \quad \langle \text{repeat}(c), \sigma'' \rangle \mapsto_c \sigma'}{\langle \text{repeat}(c), \sigma \rangle \mapsto_c \sigma'} \\
\\
\frac{\sigma(\text{flag}) = \text{ok} \quad \langle c, \sigma \rangle \mapsto_c \sigma' \quad \sigma'(\text{flag}) = \text{br}}{\langle \text{repeat}(c), \sigma \rangle \mapsto_c \sigma'[\text{ok}/\text{flag}]}
\end{array}$$

Il linguaggio FUN

Consideriamo un linguaggio di programmazione *funzionale*, che chiamiamo FUN. I programmi sono termini che contengono anche le espressioni aritmetiche e booleane. La sintassi è la seguente

$$\begin{aligned}
t ::= & x \mid n \mid \text{true} \mid \text{false} \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid \\
& t_1 = t_2 \mid t_1 < t_2 \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid \text{fun}(x, t) \mid \text{apply}(t_1, t_2)
\end{aligned}$$

dove n sono interi, $x \in \text{Var}$ sono le variabili, true e false sono le costanti booleane. Il predicato di uguaglianza è definito sugli interi, così come quello di minore.

La valutazione di un termine t avviene in uno stato (che spesso chiameremo ambiente) δ , che è una funzione da variabili Var ai valori, e restituisce un valore in un insieme $\mathcal{V}$. Tale insieme è il più piccolo insieme che contiene gli interi, i booleani e le astrazioni funzionali chiuse, le liste finite di valori e le coppie di valori. Un'astrazione funzionale chiusa è un oggetto della forma $\text{fun}(x, t)$ dove $FV(t) \subseteq \{x\}$.

L'insieme delle *free variables* del termine t , che indichiamo con $FV(t)$, è definito per induzione strutturale sui termini nel seguente modo:

$$FV(t) = \begin{cases} \{x\} & \text{se } t \text{ è } x \\ \emptyset & \text{se } t \text{ è } n, \text{true}, \text{false} \\ FV(t_1) \cup FV(t_2) & \text{se } t \text{ è } \begin{cases} t_1 + t_2, t_1 - t_2, t_1 \times t_2, t_1 \wedge t_2, t_1 \vee t_2, \\ t_1 < t_2, t_1 = t_2, \text{apply}(t_1, t_2) \end{cases} \\ FV(t') & \text{se } t \text{ è } \neg t' \\ FV(t') \setminus \{x\} & \text{se } t \text{ è } \text{fun}(x, t') \\ FV(t_0) \cup FV(t_1) \cup FV(t_2) & \text{se } t \text{ è } \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \end{cases}$$

Quindi, indicando con $\mathbb{Z}$ gli interi, $\mathbb{B}$ i booleani e con $\mathcal{F}$ le chiusure, abbiamo che $\mathcal{V} = \mathbb{Z} \cup \mathbb{B} \cup \mathcal{F}$ e questo insieme è ben definito dato che i sottoinsiemi che lo compongono sono disgiunti a due a due.

Ricapitolando abbiamo quindi che lo stato, che abbiamo detto chiamiamo ambiente, è una funzione dai nomi ai valori. In buona sostanza chiamiamo *Env* l'insieme di questi ambienti, cioè

$$Env = \{\delta : Var \rightarrow \mathcal{V} \mid \delta \text{ è una funzione parziale}\}$$

Data $\delta \in Env$, con $\delta[\mathbf{v}/x]$ denotiamo la funzione ottenuta a partire da δ e così definita: $\delta[\mathbf{v}/x](x) = \mathbf{v}$ e $\delta[\mathbf{v}/x](y) = \delta(y)$.

La semantica del linguaggio è data dalle seguenti regole:

$$\begin{array}{cccc} \overline{\langle x, \delta \rangle \rightarrow \delta(x)} & \overline{\langle n, \delta \rangle \rightarrow \mathbf{n}} & \overline{\langle \text{true}, \delta \rangle \rightarrow \mathbf{true}} & \overline{\langle \text{false}, \delta \rangle \rightarrow \mathbf{false}} \\ \\ \frac{\langle t, \delta \rangle \rightarrow \mathbf{v}}{\langle \neg t, \delta \rangle \rightarrow \neg \mathbf{v}} & \frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2}{\langle t_1 \text{ op } t_2, \delta \rangle \rightarrow \mathbf{v}_1 \text{ op } \mathbf{v}_2} & \mathbf{op}, \text{op} \in \{+, -, \times, <, =, \wedge, \vee\} & \end{array}$$

con $\mathbf{v}_1 \text{ op } \mathbf{v}_2$, dove $\text{op} \in \{+, -, \times, <, =, \wedge, \vee\}$ intendiamo che $\mathbf{v}_1$ e $\mathbf{v}_2$ sono definiti e del *tipo* corretto e quindi anche $\mathbf{v}_1 \text{ op } \mathbf{v}_2$ è definito, mentre *op* è il corrispettivo nel linguaggio di programmazione.

$$\begin{array}{cc} \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{true} \quad \langle t_1, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} & \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{false} \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \\ \\ \frac{FV(t) \subseteq \{x\}}{\langle \text{fun}(x, t), \delta \rangle \rightarrow \text{fun}(x, t)} & \frac{\langle t_1, \delta \rangle \rightarrow \text{fun}(x, t) \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2 \quad \langle t, \delta[x/\mathbf{v}_2] \rangle \rightarrow \mathbf{v}}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow \mathbf{v}} \end{array}$$

A cosa serve la semantica

Abbiamo fino ad ora visto il frammento di un linguaggio imperativo (abbiamo visto IMP e REPEAT) ed abbiamo visto un frammento di un linguaggio funzionale FUN. La formalizzazione del significato dei vari costrutti di un linguaggio di programmazione ci serve per poter parlare della *macchina astratta*. L'idea è molto semplice: è una macchina che ha il linguaggio di programmazione come *codice macchina*. Diamone una definizione:

Definizione 3 *Supponiamo sia dato il linguaggio di programmazione $\mathcal{L}$. Una macchina astratta per $\mathcal{L}$, che indichiamo con $\mathbf{M}_{\mathcal{L}}$, è un qualsiasi insieme di strutture dati ed algoritmi che permettono di memorizzare ed eseguire programmi scritti in $\mathcal{L}$.*

Se non ci interessa precisare il linguaggio, quindi siamo principalmente interessati alle strutture dati e agli algoritmi di una macchina astratta, scriviamo solo $\mathbf{M}$, omettendo il pedice che indica il linguaggio di programmazione. La definizione suggerisce che di una macchina astratta dobbiamo precisare come memorizziamo e come eseguiamo i programmi, e su come si eseguono i programmi abbiamo già visto la semantica, che però usa la nozione di stato o ambiente.

Proviamo a precisare cosa c'è nell'insieme di strutture dati ed algoritmi. Il prossimo elenco contiene alcune delle componenti che abbiamo nell'insieme.

- strutture dati ed algoritmi per la memoria: come realizziamo la *memorizzazione*;
- strutture dati ed algoritmi per un interprete, come *eseguiamo* le istruzioni;
- strutture dati ed algoritmi per il controllo, come troviamo le istruzioni da *eseguire*;
- strutture dati ed algoritmi per le operazioni primitive, come realizziamo le *operazioni* di base.

Interprete. La componente principale (ed è anche quella su cui ci concentriamo) è l'*interprete*. Un interprete serve ad eseguire un costrutto, quindi lo deve recuperare, capire cosa fa (decodificarlo), trovare eventuali operandi, eseguirlo ed eventualmente memorizzare i risultati. In sostanza un interprete non è altro che un insieme di strutture dati ed algoritmi per:

- operazioni per l'elaborazione dei dati primitivi
- operazioni e strutture dati per il controllo di sequenza di esecuzione delle operazioni
- operazioni e strutture dati per il controllo del trasferimento dati
- operazioni e strutture dati per la gestione della memoria

Linguaggio macchina. Partiamo da una macchina astratta $\mathbf{M}$, quindi di un insieme di algoritmi e strutture dati per l'esecuzione di programmi memorizzati in un certo linguaggio.

Definizione 4 *Data una macchina astratta $\mathbf{M}$, il linguaggio compreso dall'interprete di $\mathbf{M}$ è detto linguaggio macchina di $\mathbf{M}$ e lo indichiamo con $\mathcal{L}_{\mathbf{M}}$*

I programmi sono particolari dati su cui opera l'interprete ed ai componenti di $\mathbf{M}$ corrispondono componenti di $\mathcal{L}_{\mathbf{M}}$:

- tipi di dato primitivi;
- costrutti di controllo:
 - per controllare l'ordine di esecuzione, e
 - per controllare l'acquisizione e il trasferimento dati.

Un esempio molto concreto è la macchina *hardware* (astratta). Le componenti di una macchina hardware astratta sono: l'interprete (che avete visto ad ARE), la memoria, quindi i registri e le celle di memoria, dove possiamo anche memorizzare i programmi, ci sono dei registri particolari con un significato, come il *program counter* o lo *stack pointer*, ed infine abbiamo la ALU.

Implementazione.

Partiamo da una macchina astratta $\mathbf{M}$. Le sue componenti sono realizzate con strutture dati e algoritmi *implementati* nel linguaggio macchina di una macchina già esistente (implementata in qualche modo): la *macchina ospite* $\mathbf{M}_O$. Le componenti possono essere realizzate mediante:

- *hardware* (efficienza): $\mathbf{M} = \mathbf{M}_O$
- simulazione software (flessibilità): $\mathbf{M} \neq \mathbf{M}_O$
interprete di $\mathbf{M}$:
 - può coincidere con quello di $\mathbf{M}_O$: $\mathbf{M}$ è una *estensione* di $\mathbf{M}_O$
 - può essere diverso da quello di $\mathbf{M}_O$: $\mathbf{M}$ è una *realizzata* su $\mathbf{M}_O$ in modo *interpretativo*

Per poter formalizzare cosa è un'implementazione, ricordiamo che una funzione $f : A \rightarrow B$ è una corrispondenza che associa elementi da B ad elementi di A , e che funzione significa che ad ogni elemento di A associamo un elemento di B (non il vice versa). Una funzione è detta *parziale* se non è definita per tutti gli input. Un esempio di funzione parziale è il seguente

```

if x==1 then print(x);
    else while (true) do x=1;

```

se la variabile non è 1, il programma non termina (quindi per valori diversi da 1 è indefinito).

Partiamo da un linguaggio $\mathcal{L}$ che vogliamo *implementare*, quindi dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Ricordiamo che

- il linguaggio $\mathcal{L}$ ha come macchina astratta $\mathbf{M}_{\mathcal{L}}$
- la macchina astratta $\mathbf{M}$ ha come linguaggio macchina $\mathcal{L}_{\mathbf{M}}$

L'*implementazione* di $\mathcal{L}$ è la realizzazione di $\mathbf{M}_{\mathcal{L}}$ su una macchina ospite $\mathbf{M}_{\mathbf{O}}$, e la macchina ospite $\mathbf{M}_{\mathbf{O}}$ ha il suo linguaggio macchina $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$.

Un programma scritto in $\mathcal{L}$ è una funzione parziale dagli *input* agli *output*. Supponiamo di non distinguere input da output: assieme sono $\mathcal{D}$, un programma è una funzione parziale $\mathcal{P}^{\mathcal{L}} : \mathcal{D} \rightarrow \mathcal{D}$ (parziale perché potrebbe non essere sempre definita per tutti gli input, ad esempio un programma che va in *loop*).

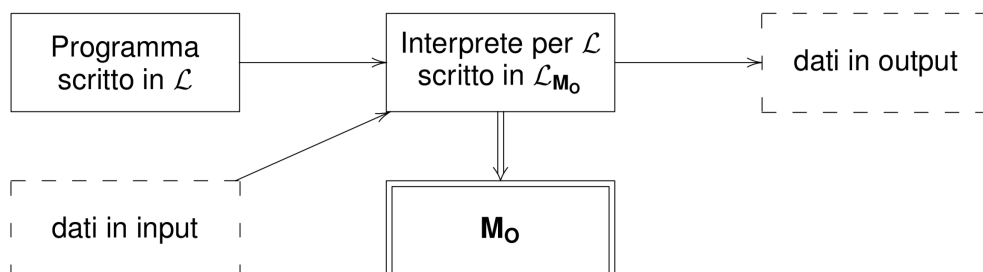
Partiamo da un linguaggio $\mathcal{L}$ che vogliamo *implementare*, e quindi dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Abbiamo la macchina ospite $\mathbf{M}_{\mathbf{O}}$ già realizzata. Scriviamo un interprete per $\mathcal{L}$ usando il linguaggio $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$, e quindi realizziamo $\mathbf{M}_{\mathcal{L}}$ usando $\mathbf{M}_{\mathbf{O}}$ ed il suo linguaggio macchina.

Per la realizzazione dell'interprete di $\mathbf{M}_{\mathcal{L}}$ osserviamo che:

- può coincidere con quello di $\mathbf{M}_{\mathbf{O}}$: $\mathbf{M}_{\mathcal{L}}$ è realizzata come *estensione* di $\mathbf{M}_{\mathbf{O}}$, ma alcune componenti di $\mathbf{M}_{\mathcal{L}}$ possono però essere diverse da quelle di $\mathbf{M}_{\mathbf{O}}$
- può essere diverso da quello di $\mathbf{M}_{\mathbf{O}}$: $\mathbf{M}_{\mathcal{L}}$ è realizzata in modo *interpretativo* su $\mathbf{M}_{\mathbf{O}}$, ma alcune componenti di $\mathbf{M}_{\mathcal{L}}$ possono coincidere con quelle di $\mathbf{M}_{\mathbf{O}}$

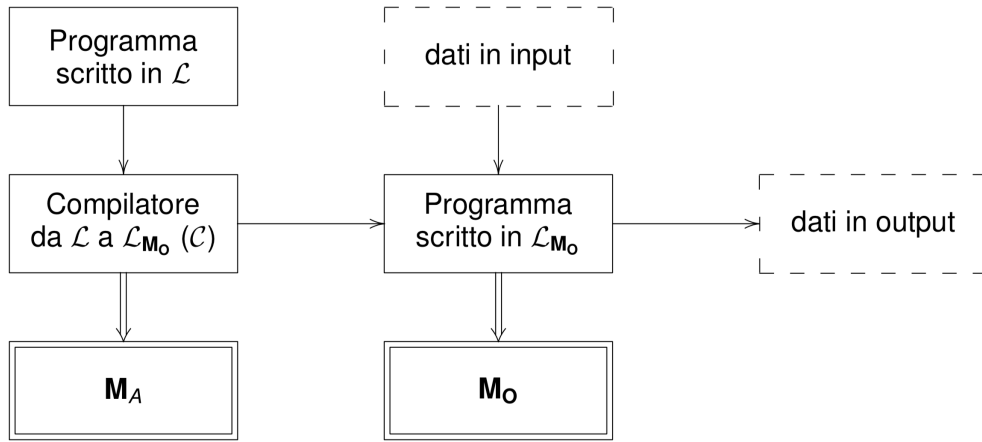
e questo a seconda della distanza tra $\mathcal{L}$ e $\mathcal{L}_{\mathbf{M}_{\mathbf{O}}}$, misurata in qualche modo.

Implementare un linguaggio: interpretazione. Prima di formalizzare cosa è un'interpretazione vediamo uno schema.



Definizione 5 Un interprete per il linguaggio $\mathcal{L}$, scritto nel linguaggio $\mathcal{L}_{\mathbf{M}_O}$, è un programma $\mathcal{I}$ che realizza una funzione parziale $\mathcal{I} : \text{Prog}^{\mathcal{L}} \times \mathcal{D} \rightarrow \mathcal{D}$ tale che $\mathcal{I}(\mathcal{P}, \text{Input}) = \mathcal{P}(\text{Input})$, dove $\mathcal{P}$ è un programma scritto in $\mathcal{L}$ e $\text{Prog}^{\mathcal{L}}$ è l'insieme dei programmi scritti in $\mathcal{L}$.

Implementare un linguaggio: compilazione. Partiamo da un linguaggio $\mathcal{L}$ che vogliamo implementare. Quindi in teoria dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Abbiamo la macchina ospite $\mathbf{M}_O$ già realizzata. Scriviamo un programma che traduce ogni programma scritto in $\mathcal{L}$ in uno funzionalmente equivalente scritto in $\mathcal{L}_{\mathbf{M}_O}$. Non realizziamo $\mathbf{M}_{\mathcal{L}}$.



Definizione 6 Un compilatore da $\mathcal{L}$ a $\mathcal{L}_{\mathbf{M}_O}$ è un programma $\mathcal{C}$ che realizza una funzione totale $\mathcal{C} : \text{Prog}^{\mathcal{L}} \rightarrow \text{Prog}^{\mathcal{L}_{\mathbf{M}_O}}$ tale che, dato un programma $\mathcal{P} \in \text{Prog}^{\mathcal{L}}$, se $\mathcal{C}(\mathcal{P}) = \mathcal{P}c$, allora per ogni input per $\mathcal{P}$ vale che $\mathcal{P}(\text{Input}) = \mathcal{P}c(\text{Input})$.

Questa traduzione è chiamata *compilazione*.

Di solito abbiamo che $\mathbf{M}_A = \mathbf{M}_O$ ma potrebbe anche non essere così, non è detto che $\mathcal{C}$ sia scritto in $\mathcal{L}_{\mathbf{M}_O}$.

Macchina Intermedia. Partiamo da un linguaggio $\mathcal{L}$ che vogliamo implementare, quindi in teoria dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}}$. Abbiamo un linguaggio intermedio $\mathcal{L}_I$ (che vogliamo implementare... quindi in teoria dobbiamo realizzare la sua macchina astratta $\mathbf{M}_{\mathcal{L}_I}$). Infine abbiamo la macchina ospite $\mathbf{M}_O$ che è già realizzata. Combiniamo i due metodi:

1. traduciamo i programmi di $\mathcal{L}$ in programmi di $\mathcal{L}_I$
2. poi interpretiamo i programmi di $\mathcal{L}_I$
3. realizziamo la macchina astratta $\mathbf{M}_{\mathcal{L}_I}$

I due casi limite ricadono in questa visione, infatti:

- interpretazione pura: $\mathbf{M}_{\mathcal{L}} = \mathbf{M}_{\mathcal{L}_I}$ e abbiamo un'implementazione puramente interpretativa
- compilazione pura: $\mathbf{M}_O = \mathbf{M}_{\mathcal{L}_I}$ e abbiamo un'implementazione puramente compilativa
- se $\mathbf{M}_O \neq \mathbf{M}_{\mathcal{L}_I}$ e $\mathbf{M}_{\mathcal{L}} \neq \mathbf{M}_{\mathcal{L}_I}$ allora:
 - se l'interprete di $\mathbf{M}_{\mathcal{L}_I}$ è sostanzialmente diverso da quello di $\mathbf{M}_O$: abbiamo un'implementazione di tipo interpretativo
non siamo nel caso della interpretazione pura perché ci sono, in $\mathcal{L}_I$, costrutti che hanno un corrispettivo diretto in $\mathcal{L}_{\mathbf{M}_O}$: non tutti i costrutti di $\mathcal{L}$ devono essere simulati
 - se l'interprete di $\mathbf{M}_{\mathcal{L}_I}$ è sostanzialmente uguale a quello di $\mathbf{M}_O$: abbiamo un'implementazione di tipo compilativo
la macchina $\mathbf{M}_O$ viene estesa con alcune funzionalità che servono per la realizzazione di costrutti di $\mathcal{L}$: il supporto a tempo d'esecuzione (**rts**: run time support)

Di solito esiste sempre una macchina intermedia. Un esempio noto a tutti: la *Java Virtual Machine* (JVM). Un programma Java viene compilato in un altro linguaggio (*bytecode*) che è il linguaggio macchina di JVM. La JVM viene realizzata in modo interpretativo sulla macchina ospite: il vostro PC (sistema operativo + chipset).

Cosa si cerca di fare?

- tradurre in un codice intermedio un linguaggio ad alto livello
- codice intermedio possibilmente ad alto livello (un esempio? il C)
- interpretazione del codice intermedio, in generale cercando di utilizzare lo stesso interprete della macchina ospite

Supporto a tempo d'esecuzione. Cosa è esattamente? Citiamo prima alcuni esempi pratici:

- il caso “paradigmatico”: il FORTRAN (che è praticamente una notazione *ad alto livello* per un linguaggio macchina)
- un caso “contemporaneo”: librerie OPENCL e macchina astratta **GPU** (Graphics Processing Unit)
macchina astratta dedicata solo per la grafica, ma ha un suo linguaggio macchina e la si *programma*!
- un caso oramai classico: il compilatore C.

Fortran. In linea di principio, è possibile tradurre completamente un programma FORTRAN in un linguaggio macchina puro, senza chiamate al rts, ma la traduzione di alcune primitive FORTRAN (per esempio, relative all'I/O) produrrebbe centinaia di istruzioni in linguaggio macchina, quindi:

- se le inserissimo nel codice compilato, la sua dimensione crescerebbe a dismisura
- in alternativa, possiamo inserire nel codice una chiamata ad una routine (indipendente dal particolare programma)
- tale routine deve essere “caricata” su $\mathbf{M_O}$ ed entra a far parte del rts.

OpenCL. Nel caso di OPENCL, bisogna scrivere un programma per una **GPU**. Si potrebbe ogni volta scrivere tutto ciò che serve, ma

- possiamo inserire nel codice una chiamata a routine (indipendente dal programma) che realizza quello che mi serve
- “caricata” sulla $\mathbf{M_O}$ (**GPU**): entra a far parte del rts

C. Il C è un linguaggio di programmazione per alcuni versi simile al FORTRAN, ha molta notazione ad *alto livello* per cosa sta sotto, ma ha anche caratteristiche moderne per la gestione delle chiamate di procedura o della memoria.

Il compilatore C produce codice per una macchina intermedia in cui il supporto a tempo di esecuzione contiene

- varie strutture dati:
 - per la pila dei records di attivazione (ad esempio, l’ambiente, la memoria, i sottoprogrammi)
 - per la memoria a heap (ad esempio, i puntatori)
- i sottoprogrammi che realizzano le operazioni necessarie su tali strutture dati

il codice prodotto è scritto in linguaggio macchina esteso con chiamate al rts.

Valutazione parziale. Oltre alla compilazione (traduzione di un programma scritto in un linguaggio in un programma scritto in un altro linguaggio), abbiamo altre tecniche di trasformazione di un programma. $P(X, Y)$ è un programma P che elabora prima i dati X e poi, a seconda del risultato, i dati Y . Se i dati X sono *comuni* a molte esecuzioni, trasformo $P(X, Y)$ in un programma $P'(Y)$ fissando i dati X . Ovviamente $P'(Y)$ è più veloce.

Definizione 7 $\mathcal{P}eval_{\mathcal{L}} : (Prog^{\mathcal{L}} \times \mathcal{D}) \rightarrow Prog^{\mathcal{L}}$ è tale che, dato un programma P scritto in $\mathcal{L}$ e che opera su due argomenti, allora $\mathcal{P}eval_{\mathcal{L}}(P, D) = P'$ tale che, qualsiasi sia un input Y , abbiamo che $\mathcal{I}_{\mathcal{L}}(P(D, Y)) = \mathcal{I}_{\mathcal{L}}(P'(Y))$ dove $\mathcal{I}_{\mathcal{L}}$ è l’interprete di $\mathcal{L}$.

SECD

Presentiamo la prima formalizzazione di macchina astratta, la SECD. La SECD (Stack, Environment, Control, Dump) è una macchina astratta per un linguaggio funzionale. Il linguaggio è il seguente:

$$t ::= \mathbf{c}_i \mid \mathbf{var}(x) \mid op_i(t_1, \dots, t_{a_i}) \mid \mathbf{if}(t_1, t_2, t_3) \mid \mathbf{fun}(f, x, t) \mid \mathbf{apply}(t_1, t_2)$$

dove $\mathbf{c}_i$ sono le costanti del linguaggio, op_i sono un numero di operazioni primitive, ognuna con la propria arietà (numero di argomenti), mentre il resto è standard.

La macchina stessa è una pila dove ogni elemento della pila è una tripla (S, E, C) dove S è una pila di risultati, E è un ambiente, che ora è una funzione da nomi (identificatori) a valori denotabili (scopriremo nel seguito cosa sono), e C è una pila di elementi da *valutare* (il nostro programma memorizzato).

Dato che abbiamo i *valori denotabili*, dobbiamo sostanzialmente dire quali sono: interi, booleani e *funzioni* ($\mathbf{fun}(f, x, t)$).

Vediamo più nel dettaglio come sono fatte le strutture dati. La S è una pila di valori denotabili (ogni valore lo indichiamo con v), mentre E è una funzione che associa ad ogni variabile x (identificatore) la sua *denotazione*. Da ultimo C è una pila di elementi di controllo, dove un elemento di controllo obbedisce alla seguente grammatica:

$$ce ::= t \mid \text{Prim}(op_i) \mid \text{App} \mid \text{If}$$

le $\text{Prim}(op_i)$ sono la controparte implementativa delle varie op_i sintattiche e $\tilde{op}_i$ sono la controparte *semantica* di queste.

Lo *stato iniziale* della macchina è $([\], \perp, t :: [\]) :: [\]$ dove $\perp$ è la funzione ovunque indefinita (o se volete quella che ad ogni variabile x associa un valore *undefined*), mentre lo *stato finale* della macchina è $(v :: [\], \perp, [\]) :: [\]$

Diamo ora di seguito le regole che descrivono l'interprete. Possiamo immaginare che queste regole trasformano la macchina astratta in una nuova macchina astratta.

$$(S, E, op_i(t_1, \dots, t_{a_i}) :: C) :: D \mapsto (S, E, t_1 :: \dots :: t_{a_i} :: \text{Prim}(op_i) :: C) :: D$$

$$(S, E, \mathbf{if}(t_1, t_2, t_3) :: C) :: D \mapsto (S, E, t_1 :: \text{If} :: t_2 :: t_3 :: C) :: D$$

$$(S, E, \mathbf{apply}(t_1, t_2) :: C) :: D \mapsto (S, E, t_1 :: t_2 :: \text{App} :: C) :: D$$

$$(S, E, \mathbf{n} :: C) :: D \mapsto (n :: S, E, C) :: D$$

$$(S, E, \mathbf{b} :: C) :: D \mapsto (b :: S, E, C) :: D$$

$$(S, E, \mathbf{var}(x) :: C) :: D \mapsto (E(x) :: S, E, C) :: D$$

$$(S, E, \mathbf{fun}(f, x, t) :: C) :: D \mapsto (\mathbf{fun}(f, x, t) :: S, E, C) :: D$$

$$(v_2 :: v_1 :: S, E, \text{App} :: C) :: D \mapsto ([\], E[v_1/f, v_2/x], t :: [\]) :: (S, E, C) :: D \text{ dove } v_1 = \mathbf{fun}(f, x, t)$$

$$\begin{aligned}
(v :: S, E, []) :: (S', E', C') :: D &\mapsto (v :: S', E', C') :: D \\
(v_{a_i} :: \dots :: v_1 :: S, E, \text{Prim}(op_i) :: C) :: D &\mapsto (\tilde{op}_i(v_1, \dots, v_{a_i}) :: S, E, C) :: D \\
(true :: S, E, \text{If} :: t_2 :: t_3 :: C) :: D &\mapsto (S, E, t_2 :: C) :: D \\
(false :: S, E, \text{If} :: t_2 :: t_3 :: C) :: D &\mapsto (S, E, t_3 :: C) :: D
\end{aligned}$$

Osservate che la macchina medesima è una pila che è sostanzialmente la pila dei record d'attivazione.

Ambiente e memoria

In un linguaggio di programmazione abbiamo, in generale, dei *nomi* che distinguiamo dalle parole chiave (keywords) che sono proprie del linguaggio. Nel seguente frammento di programma

```
int pippo (x : int) {
  while x > 10 do x = x - 2;
}
```

In questo frammento di programma ci sono delle *keywords*, ad esempio `int`, `while`, `'` e altre, ma ci sono anche dei *identificatori* come `pippo` e `x` oppure altre sequenze di caratteri come `10` e `2` che indicano oggetti a cui non bisogna dare particolare interpretazione. La grammatica con cui vengono generati i nomi non ci interessa particolarmente, anche se ci possono essere delle regole particolari per capire cosa è associato a quel nome. Un nome lo si usa per identificare un qualche oggetto (valore o altro) che altrimenti non sapremmo identificare, quindi immaginiamo ci siano anche degli oggetti che chiameremo in generale *valori*.

Diamo una definizione di insieme dei nomi di un linguaggio.

Definizione 8 Dato un linguaggio di programmazione $\mathcal{L}$, con $\text{Name}_{\mathcal{L}}$ indichiamo l'insieme dei nomi di quel linguaggio.

Questa definizione ci servirà in seguito. Facciamo alcuni esempi:

- `var pippo`: la sequenza di caratteri in rosso denota una variabile mentre la sequenza di caratteri in blu è una keyword del linguaggio
- `const pi = 3.14`: la sequenza di caratteri in rosso denota la costante 3.14
- `int fib`: la sequenza di caratteri in rosso denota una funzione

Nei linguaggi i nomi sono spesso *identificatori* (token alfa-numeric), sono nomi anche le *keywords* o simboli come `+`, `:=`, `++`, ecc., che non sono identificatori ma fanno lo stesso parte del linguaggio.

Un nome serve ad indicare in generale un oggetto, che è l'oggetto *denotato*. I nomi sono gli oggetti simbolici più facili da ricordare (buona regola di programmazione) e, particolarmente importante, permettono l'astrazione (sia sui dati che sul controllo). Ricordiamo che astrarre significa spesso dimenticare dettagli non importanti, che però in qualche forma sono presenti. Il non importanti significa che su quelli non ci concentriamo, mentre ci concentriamo sull'uso che si fa dell'astrazione.

Oggetti denotabili

Diamone prima una definizione molto generale.

Definizione 9 *Un oggetto denotabile è un oggetto a cui possiamo dare un nome.*

Osserviamo che non necessariamente diamo nome a tutto, ad esempio, in generale, non diamo nomi alla memoria.

Ci sono i nomi definiti dall'utente come le variabili, parametri formali, le procedure o funzioni, i tipi definiti dall'utente, le etichette, i moduli, le costanti definite dall'utente e simili, mentre ci sono i nomi definiti dal linguaggio come i tipi primitivi, le operazioni primitive, le costanti predefinite e così via.

Possiamo ora definire cosa è l'*ambiente*.

Definizione 10 *L'ambiente è un insieme di associazioni tra nomi ed oggetti denotabili esistenti a tempo d'esecuzione (run-time) in uno specifico punto del programma ed in un specifico momento dell'esecuzione*

L'ambiente è anche chiamato *referencing environment*. Collegate alla nozione di ambiente abbiamo varie nozioni che è opportuno sottolineare. La prima è quella ovvia di *binding*, ossia di legame, o anche associazione, che è quella che abbiamo tra nomi ed oggetti denotati, la seconda, altrettanto rilevante, è quella di *binding time*, che è il momento in cui avviene il *binding*, ed infine abbiamo la nozione di *scope*, che è la parte di programma in cui una particolare associazione tra nome e oggetto denotato, cioè un *binding* è valida. Lo scope ha come sinomimi visibilità, campo d'azione, e portata.

Definizione 11 *Dato un programma ed un dato nome, lo scope di una certa associazione tra il nome e l'oggetto denotato da quel nome, è dato dalla parte di programma in cui quell'associazione è nota a tempo d'esecuzione.*

In altre parole lo scope serve per capire come è fatto l'ambiente in un certo punto del programma.

Ci domandiamo ora come formare (o inserire) un'associazione tra nome ed oggetto denotabile in un ambiente. In generale lo facciamo esplicitamente con una *dichiarazione*.

Definizione 12 *La dichiarazione è un meccanismo, implicito o esplicito, con cui si crea un'associazione nell'ambiente.*

Il modo esplicito è quello che ci immaginiamo: all'interno del programma troviamo parti come `int x = 3`, `int f ()` e simili, e quando si incontrano queste dichiarazioni si crea un'associazione nell'ambiente tra il nome e l'oggetto denotato, che in generale viene creato al momento della dichiarazione. Il modo implicito lo abbiamo principalmente quando abbiamo programmi divisi in moduli, in questo caso l'associazione si crea al momento dell'uso di un nome, che è dichiarato in un'interfaccia.

Il momento in cui avviene il legame, cioè il *binding time* ha particolare rilevanza. Può essere *statico*:

- al momento della progettazione del linguaggio: tipi primitivi, nomi per operazioni e costanti predefinite,
- al momento della scrittura del programma: definizione di alcuni nomi, per esempio variabili, funzioni, il cui legame potrà essere completato dopo (definizioni di interfacce),
- al momento, se c'è, della compilazione: il legame di alcuni nomi, ad esempio le variabili globali, può essere determinato in questa fase.

Oppure può essere *dinamico*:

- al momento dell'esecuzione: legame definitivo di tutti i nomi non ancora legati, come le variabili locali ai blocchi

Non va confuso con lo scope: sono due fenomeni diversi.

Ci sono dei fenomeni particolari che è utile sottolineare. Lo stesso nome può denotare oggetti distinti o in *punti* diversi del programma (vedremo che sono in fondo ambienti diversi) o in *momenti* diversi (ad esempio, procedure ricorsive) Oppure nomi diversi possono denotare lo stesso oggetto (*aliasing*). Il seguente programma (C-like) ne fornisce un esempio:

```
int *X, *Y;           // X, Y puntatori ad interi
X = (int *) malloc (sizeof (int));
                      // allocata la memoria puntata
*X = 5;               // dereferenziazione (e assegnamento)
Y = X;                // X e Y puntano allo stesso oggetto
*Y = 10;
write(*X);             // stampa 10
```

Ambiente: come si organizza

La nozione centrale per l'organizzazione dell'ambiente è quella di blocco.

Definizione 13 *Un blocco è regione testuale del programma, identificata da un segnale di inizio ed uno di fine, che può contenere dichiarazioni locali a quella regione.*

I blocchi sono o *anonimi* (o in-line) che significa che alla regione non è associato alcun nome, oppure possono non essere anonimi e quindi sono in generale associati ad una procedura o funzione. In questo modo il blocco ha un nome, quello della procedura o della funzione. Per identificare i blocchi anonimi abbiamo bisogno di alcuni *segnali*. In Algol abbiamo `begin ... end`, in C o Java abbiamo le parentesi graffe `{ ... }`, in Lisp abbiamo le parentesi tonde `( ... )` ed in ML invece il `let ... in ....` Per convenzione le dichiarazioni sono subito dopo il segnale di inizio del blocco, in ML sono tra il `let` e l'`in`.

Un blocco può essere anche visto come un modo per combinare una serie di istruzioni facendole vedere all'esterno come un'unica istruzione (con effetti che sono quelli cumulati). Nel caso della sintassi astratta sarà facile capire cosa sta in un blocco, dato avremo un costruttore di blocco. Inoltre il blocco consente la gestione locale dei nomi (quelli dichiarati nel blocco sono locali al blocco). Questo aumenta la chiarezza e si possono scegliere i nomi più adatti e significativi. Inoltre i blocchi consentono di migliorare l'occupazione di memoria e permettono la ricorsione. Infine i blocchi possono essere solo annidati (uno dentro l'altro) e mai parzialmente sovrapposti (la fine del primo si trova prima della fine del secondo):

- `begin ... begin ... end ... end`
in questo caso il blocco rosso e quello blu si sovrappongono parzialmente e questo non è consentito
- si: `begin ... begin ... end ... end`
in questo caso il blocco blu è totalmente contenuto nel blocco rosso e questo è consentito.

Tipi d'ambiente e operazioni

In effetti c'è solo un ambiente, come la definizione di ambiente suggerisce, ma per convenienza introduciamo delle distinzioni:

- ambiente *locale*: sono le associazioni create all'ingresso nel blocco (variabili locali, parametri formali, ecc.),
- ambiente *non locale*: sono le associazioni create fuori dal blocco ma che sono ancora valide all'interno di quel blocco,
- ambiente *globale*: sono le associazioni che sono comuni a tutti i blocchi,

quindi l'ambiente si suddivide in questi tre tipi. Un ambiente ha sempre la parte locale, mentre può avere la parte globale o non locale. Osserviamo che l'ambiente globale è un caso particolare di ambiente non locale.

Elenchiamo ora quali sono le operazioni che si compiono sull'ambiente.

- *creazione*: creo un'associazione nome - oggetto denotato (tipicamente in ingresso ad un blocco, quando si incontrano le dichiarazioni,

- *referimento*: uso in qualche modo l'oggetto denotato attraverso il suo nome,
- *disattivazione*: un'associazione viene disattivata (ad esempio, in un blocco interno uso un nome usato in un blocco più esterno),
- *riattivazione*: un'associazione disattivata ritorna in *vita* (ad esempio, esco dal blocco interno),
- *distruzione*: l'associazione nome - oggetto denotato viene cancellata definitivamente (esco dal blocco dove l'associazione è stata creata).

Osserviamo che le operazioni sull'ambiente riguardano la creazione, uso e distruzione di un legame, legame tra un nome trovato nel testo del programma ed un oggetto. Fino ad ora abbiamo considerato dell'ambiente solo il legame ed il nome, e tramite il legame ed il nome siamo in grado di accedere o usare l'oggetto denotato. Ci domandiamo quali operazioni si possono compiere su un oggetto.

- la *creazione* dell'oggetto, possibilmente prima o al momento della creazione del legame (per creare un legame devo avere l'oggetto),
- l'*accesso* all'oggetto, facendovi riferimento,
- la *modifica* dell'oggetto, attraverso il nome con cui vi accediamo e con le opportune operazioni che lo *modificano* l'oggetto ma non il nome ed il legame,
- la *distruzione* dell'oggetto.

Combinando le operazioni su ambiente e oggetti abbiamo il seguente schema:

1. creazione di un oggetto
2. creazione di un legame per l'oggetto (ambiente)
3. accesso e/o modifica dell'oggetto (attraverso il legame)
4. disattivazione di un legame
5. riattivazione di un legame
6. distruzione di un legame
7. distruzione dell'oggetto

La *vita* dell'oggetto va da 1 a 7, mentre la *vita* dell'associazione va da 2 a 6.

La vita di un oggetto in generale non coincide con quella dei legami per quell'oggetto:

- *vita lunga*: la vita dell'oggetto è più lunga di quella del legame (ad esempio, un parametro passato ad una procedura per *referimento*, che vedremo è il modo più diffuso)

- vita *corta*: la vita dell'oggetto è più corta di quella del legame.

Mentre la prima è una situazione naturale, la seconda non lo è. Vediamo un esempio:

```
int *X, *Y;
...
X = (int *) malloc (sizeof (int));
Y = X;
...
free(X);      // deallocazione memoria (distruzione oggetto)
```

in questo caso il legame tra Y e la memoria esiste sempre anche se non c'è la memoria che è stata deallocata. Il fenomeno è quello dei *dangling reference*.

Una volta creato un nome ed un legame, dove può essere usato? Cioè dove è noto sia il nome che il legame? Abbiamo una regola generale:

Definizione 14 *Una dichiarazione locale ad un blocco è visibile in quel blocco e in tutti i blocchi in esso annidati, a meno che non intervenga in tali blocchi una nuova dichiarazione dello stesso nome (che nasconde, o maschera, la precedente).*

Come interpretare la regola in presenza di procedure/funzioni e come interpretarla in presenza di ambiente non locale (e non globale)?

Cerchiamo di precisare il problema. Consideriamo il seguente programma (di cui molte parti sono omesse) scritto, come molti, in stile imperativo.

```
{int x = 10;
...
void uno () {
    x++;
}
...
void due () {
    int x = 2;
    uno();
}
...
due();
}
```

Il nome x viene definito in due blocchi. Il primo blocco in cui viene dichiarata è un blocco anonimo mentre il secondo blocco è quello con un nome (la procedura **due**).

Nell'esecuzione del programma si effettua una chiamata alla procedura **due**. In questo caso quale x viene incrementata?

Si possono adottare due strategie: la prima è far riferimento al nome dichiarato nel blocco più vicino che racchiude testualmente il blocco dove il nome viene usato, la seconda è fare riferimento al legame attivo al momento della chiamata della procedura/funzione.

- scope *statico*: si fa riferimento al legame creato nel blocco testualmente più vicino
- scope *dinamico*: si fa riferimento al legame attivo

sostanzialmente si deve decidere in quale ambiente risolvere il legame: nel caso dello scope *statico* nell'ambiente determinato dalla scrittura del programma, nel caso dello scope *dinamico* nell'ambiente determinato dall'esecuzione del programma.

Scope statico

Il nome è risolto nel blocco che testualmente lo racchiude:

```
{int x = 10;
void pippo (int n) {
    x = n+1;}
pippo(3);
write(x);    // stampa 4
    {int x = 2;
      pippo(3); // qui si modifica la x esterna al blocco
      write(x); // stampa 2, la x interna non viene modificata dalla pippo
    }
write(x);    // stampa 4
}
```

Scope dinamico

In questo caso il nome è risolto nel blocco attivato e non ancora disattivato:

```
{int x = 10;
void pippo (int n) {
    x = n+1;}
pippo(3);
write(x);    // stampa 4
    {int x = 2;
      pippo(3); // qui si modifica la x del blocco anonimo interno
      write(x); // stampa 4, la x interna viene modificata dalla pippo
    }
write(x);    // stampa 4
}
```

Scope statico o scope dinamico?

Quasi tutti i linguaggi di programmazione hanno lo scope statico, dato che lo scope statico soddisfa i principi di *indipendenza* dalla posizione dell'uso del nome: una procedura può essere chiamata in contesti diversi ma importa solo dove è stata dichiarata, e di *indipendenza* dai nomi locali: se si modifica un nome locale non si ripercuote sulla procedura. Inoltre lo scope statico consente di *specializzare* le funzioni/procedure.

Vediamo, con due esempi, i principi di indipendenza dello scope statico.

Indipendenza dalla posizione

```
{int x = 10;
void uno () {
    x++;}
void due () {
    int x = 2;
    uno();}
due();
uno()
}
```

La procedura **uno** viene usata in contesti diversi: all'interno della **due** e nel blocco in cui è dichiarata, ma non importa dove si usa, la **x** a cui fa riferimento è sempre la stessa, quella dichiarata nel blocco anonimo. Se lo scope fosse dinamico, questo non sarebbe vero.

Indipendenza dai nomi locali

```
{int x = 10;
void uno () {
    x++;}
void due () {
    int y = 2;
    uno();}
due();
uno()
}
```

Se cambio la **y** in **due** con **x** non succede nulla, ma se lo scope fosse dinamico allora la semantica cambierebbe.

Specializzazione

Anche lo scope dinamico obbedisce ad un principio, quello di *specializzazione*. Consideriamo una procedura che *colora* il video ed ha una variabile **colore**.

```

{...
void visualizza(text x) {
    ...
    printscreen(x,colore);
}
colore = rosso;
visualizza("prova",colore);
}

```

Questa visualizza sullo schermo la parola “prova” su sfondo rosso.

Scope statico e dinamico

Confrontiamo un po’ i due approcci allo scope, cercando di evidenziarne i vantaggi e gli svantaggi. Nel caso dello scope statico (detto anche scoping statico, statically scoped, lexical scope) abbiamo

- l’informazione è completa dal testo del programma
- le associazioni sono note a tempo di compilazione
- valgono i principi di indipendenza
- è complesso da implementare (lo vedremo), ma in generale efficiente
- i principali linguaggi lo implementano: Algol, Pascal, C, Java, ML, ecc

Nel caso dello scope dinamico (o scoping dinamico, dynamically scoped), abbiamo invece

- l’informazione è derivata dall’esecuzione
- i programmi con scope dinamico sono meno “leggibili”
- è semplice da implementare, ma meno efficiente
- pochi linguaggi, uno è Lisp (alcune versioni), quello più usato è Perl.

I due tipi di scope differiscono solo in presenza congiunta di ambiente non locale e non globale, quindi certamente con le procedure o le funzioni, ma anche in assenza di ambiente globale che coinvolge tutti i nomi non locali.

Ci sono eccezioni e problemi.

Cosa succede se scrivo in Pascal

```

const pippo = valore;
const valore = 0;

```

dovrei derogare alla regola che dice che un nome può essere usato solo dopo che è stato definito, ma se lo inserisco in un contesto come

```

const valore = 1;
procedure pluto
begin
  const pippo = valore;
  const valore = 0;
  ...
end

```

riesco a dare il valore alla costante, che poi ridefinisco, quindi ora la costante `valore` dovrebbe contenere 0, ma questa dichiarazione di `valore` arriva dopo il suo effettivo uso. Sempre in Pascal la dichiarazione di una lista viene fatta in questo modo

```

type lista = ^elemento;
type elemento record
  info: integer;
  next: lista
end

```

e quindi il tipo `lista` è definito in funzione del tipo `elemento` che contiene un riferimento al tipo `lista`. In questi casi si decide di aspettare per capire come risolvere il nome. Lo stesso fenomeno si ha con le procedure mutuamente ricorsive.

Altri linguaggi consentono dichiarazioni *incomplete* (esempio, ADA):

```

type elemento;
type lista = access elemento;
type elemento is record
  info: intero;
  next: lista
end

```

Qui la prima dichiarazione è incompleta, e si adotta la stessa strategia che si usa con la mutua ricorsione in Pascal.

Il C ha lo scope statico e non permette di definire funzioni in blocchi interni. Quindi l'ambiente di una funzione è diviso in ambiente locale e globale e la parte non locale e non globale non esiste. I nomi non locali vengono sempre risolti nell'ambiente più esterno (quello globale).

In C realizzo lo scope dinamico con la direttiva `#define` che corrisponde ad una macro espansione

```

int x=10;
int next_x(int delta) { return x + delta; }
#define NEXT_X(delta) x+delta
main(){
  int x=5;
  printf("%d, %d\n", next_x(4), NEXT_X(4));
}

```

la chiamata `NEXT_X(4)` nel `main` viene inserita nel codice.

In Java in generale una variabile è visibile a partire dalla dichiarazione e fino alla fine del blocco, e solo dopo la dichiarazione di una variabile, quindi il seguente frammento di codice non è ammesso

```
{a=1;
  int a;
  ...
}
```

mentre il seguente si

```
{void f(){
    g();
}
void g(){
    f();
}
...
}
```

Java permette metodi mutuamente ricorsivi.

Un ulteriore problema in Pascal si ha con dichiarazioni “normali” e mutuamente ricorsive.

```
procedure pippo;
...
end (*pippo*)
procedure A;
...
  procedure B
  begin
    ...
    pippo;
    end (*B*)
  ...
  procedure pippo;
  begin
    ...
    end)
```

quale `pippo` devo usare in `B`? Il compilatore non sa decidere.

L’ambiente è l’insieme di legami tra nomi ed oggetti denotabili e le regole di scope determinano come i legami vengono ereditati tra i vari ambienti. Ci sono poi delle regole specifiche per determinare lo scope di nomi in situazioni particolari (alcune situazioni le abbiamo viste, altre le vedremo), per il passaggio dei parametri e per il *binding*, che può essere *shallow* o *deep*, di parametri formali ai quali si associano funzioni.

Ambiente locale dinamico

Ricordiamo che l'ambiente è sempre locale (può anche essere globale o non locale), e in effetti ci riferiamo a quelle locali, non locali o globali (appartenenti ad un solo blocco, comuni a vari blocchi, comuni a tutti i blocchi). La creazione di una associazione tra nome e oggetto denotato avviene in generale all'ingresso di un blocco, e viene inserita nell'ambiente, mentre la distruzione di una associazione tra nome e oggetto denotato avviene all'uscita dal blocco in cui l'associazione è stata creata e viene eliminata dall'ambiente.

L'associazione è utilizzabile solo all'interno del blocco (a parte eventuali buchi perché siamo entrati in un altro blocco) e se rientriamo nello stesso blocco, rieseguiamo le dichiarazioni e creiamo *nuove* associazioni. Lo chiamiamo *dinamico* perché ogni volta creiamo nuove associazioni.

Ambiente locale statico

In questo caso non creiamo associazioni quando si entra in un blocco, ma all'ingresso di questo avviene la riattivazione di un'associazione tra nome e oggetto denotato nell'ambiente, e all'uscita del blocco in cui l'associazione è stata attivata, avviene la disattivazione dell'associazione nell'ambiente. Inoltre l'associazione è utilizzabile solo all'interno del blocco, cioè quando è attiva, e se rientriamo nello stesso blocco, non rieseguiamo le dichiarazioni, ma ci limitiamo a riattivare le associazioni precedenti. La domanda è quindi chi esegue le dichiarazioni, e queste sono eseguite in generale o dal compilatore (prima dell'inizio dell'esecuzione) oppure alla prima esecuzione del blocco (o simile).

Nei moderni linguaggi di programmazione l'ambiente locale è dinamico, ma alcuni linguaggi hanno (anche) ambiente locale statico. In particolare nel Fortran dove non ci sono blocchi, questa è la regola per l'ambiente locale dei sottoprogrammi, le diverse attivazioni (chiamate) dello stesso sottoprogramma condividono ambiente e memoria locali, e quindi la chiamata *n*-esima trova lo stato lasciato dalla chiamata (*n*-1)-esima. Nell'Algol, nel PL/I o nel C alcune dichiarazioni locali (a blocchi o sottoprogrammi) possono essere dichiarate statiche (**static**, **own**, ...) e per queste particolari associazioni l'ambiente è statico, le altre sono trattate con l'ambiente dinamico. In Java le dichiarazioni **static** all'interno di una classe provocano la creazione di ambiente (ed eventuale memoria) che appartengono alla classe e non agli oggetti sue istanze, sono eseguite una sola volta all'atto della esecuzione della dichiarazione di classe e possono costituire uno stato interno comune a tutti gli oggetti della classe.

Perché usare l'ambiente locale statico? Le dichiarazioni statiche sono più efficienti, ma creano comportamenti molto più complessi da comprendere dal punto di vista semantico dato che la presenza di uno stato interno fa sì che la semantica debba tener conto di tutta la sequenza di attivazioni. Queste sono utili se si vogliono definire blocchi (o meglio sottoprogrammi) dotati di stato interno (ad esempio, ungeneratore di numeri casuali, o un generatore di nomi sempre nuovi, ecc.), inoltre l'implementazione dell'ambiente locale statico è complessa.

Gestione della memoria

La memoria può essere *statica*, cioè la memoria viene allocata a tempo di compilazione (ad esempio, le variabili globali), oppure *dinamica*, ad esempio la memoria viene allocata o su una *pila* (stack), dove tutti gli oggetti sono allocati con politica LIFO, oppure su uno *heap*, dove gli oggetti sono allocati e deallocati in qualsiasi momento (in generale per mezzo di puntatori). Inoltre l'implementazione delle regole di scope è legata all'avere sottoprogrammi e/o meccanismi come blocchi per strutturare l'ambiente (se non li avessimo, la gestione della memoria sarebbe ovviamente sempre statica), non avremmo necessità di realizzare le regole di scope.

Allocazione statica

In questo caso un oggetto ha un *indirizzo assoluto* che è mantenuto per tutta l'esecuzione del programma.

Solitamente sono allocate staticamente le variabili *globali*, le variabili *locali* di sottoprogrammi senza ricorsione, le costanti determinabili a tempo di compilazione e le tabelle usate dal supporto a run-time (per type checking, garbage collection, ecc.). Spesso sono usate zone protette di memoria.

Va osservato che l'allocazione statica ha come conseguenza che non è possibile avere allo stesso tempo incarnazioni diverse della stessa funzione o porzione di programma, quindi la ricorsione non è sostanzialmente implementabile, a parte la ricorsione in coda per motivi che saranno chiari successivamente.

Allocazione dinamica: la pila

Ogni istanza di un sottoprogramma a *run-time* ha una porzione di memoria detta *record di attivazione* (RdA o frame) contenente le informazioni relative alla specifica istanza (compreso l'indirizzo di ritorno). Anche ogni blocco anonimo ha un suo record di attivazione (che è più semplice). La pila (LIFO) è la struttura dati *naturale* per gestire i RdA perché le chiamate di procedura (anche ricorsive) ed i blocchi sono annidati uno dentro l'altro.

Vediamo nel dettaglio come viene gestita la pila dei RdA, precisando prima cosa in generale contiene un generico RdA. Un RdA contiene tutte le informazioni relative alla specifica istanza del blocco in esecuzione, quindi il record d'attivazione di un blocco anonimo ha spazio per i nomi *locali*, quelli cioè che sono dichiarati nel blocco (e che serviranno a capire cosa è associato a quel nome, quindi l'ambiente locale), eventuale spazio per memorizzare risultati intermedi, cioè quelli che sono i risultati parziali della valutazione di un'espressione complessa (ad esempio $(x + y)/(2 * x) + x * y$ ha bisogno di memorizzare i valori delle sotto espressioni $(x + y)/(2 * x)$ e $x * y$), il riferimento al RdA del blocco (anonimo o no) che *racchiude* il blocco che stiamo eseguendo, cioè quello in cima alla pila degli RdA, e che si chiama in generale *puntatore di catena dinamica*, ed il *puntatore di catena statica*. La catena statica consente di identificare il record d'attivazione che contiene l'ambiente non locale nel caso di scope statico. Il record

d'attivazione di un blocco non anonimo, quindi di una procedura o funzione, contiene, oltre a quello che contiene il RdA di un blocco anonimo, anche spazio per i parametri, che fanno parte dell'ambiente locale del blocco, informazioni sull'*indirizzo* di ritorno, e l'*indirizzo* dove mettere il risultato se è una funzione che restituisce un risultato.

Nella gestione della pila dei RdA identifichiamo i passi:

- *sequenza di chiamata*: è il codice eseguito dal chiamante immediatamente prima della chiamata, in questa fase si prepara il RdA che deve essere messo sulla pila
- *prologo*: è il codice eseguito all'inizio del blocco (con nome o senza), e qui si eseguono le dichiarazioni e se serve si effettuano le associazioni per il passaggio dei parametri
- *epilogo*: è il codice eseguito alla fine del blocco (con nome o senza), che prepara all'effettivo *pop* dalla pila del record d'attivazione,
- *sequenza di ritorno*: è il codice eseguito dal chiamante immediatamente dopo la chiamata, e coinvolge non solo il levare il RdA dalla cima della pila, ma anche la comunicazione dei risultati e delle modifiche effettuate dal blocco che era in cima alla pila, cioè in esecuzione.

L'indirizzo in memoria di un RdA non può essere noto a compile-time, quindi c'è un *Puntatore RdA* (o *SP*, *stack pointer*) che punta al RdA del blocco attivo e le informazioni contenute in un RdA (nomi locali, risultati intermedi, ecc.) sono accessibili per offset rispetto allo SP:

$$\text{indirizzo-info} = \text{contenuto(SP)} + \text{offset}$$

e l'*offset* è determinabile staticamente.

Memoria e pila degli RdA

Come abbiamo sempre un ambiente *locale* abbiamo anche sempre una memoria *locale*, e questa è *allocata* nel RdA. In particolare troviamo tutta la memoria relativa alle strutture dati statiche (la cui dimensione è determinata al momento della compilazione o scrittura del programma) è allocata nel RdA, mentre nel caso di memoria relativa a strutture dati dinamiche, la cui dimensione è determinata dall'esecuzione, nel RdA si alloca solo il modo per arrivarci (un altro puntatore).

Ambiente e pila degli RdA

I record d'attivazione servono anche ad implementare l'*ambiente* (alla memoria ci accederò normalmente con un nome, e quindi devo capire soprattutto come si *implementano* le regole di scope).

Ricordiamo che abbiamo una regola di visibilità che dice

Una dichiarazione locale ad un blocco è visibile in quel blocco e in tutti i blocchi in esso annidati, a meno che non intervenga in tali blocchi una nuova dichiarazione dello stesso nome (che nasconde, o maschera, la precedente).

se una dichiarazione è *locale* ad un blocco, come la troviamo se siamo in un altro blocco? Per usarla non possiamo usare l'offset e l'SP ma dobbiamo *risalire*, all'interno della pila, per trovare il giusto RdA dove la variabile è dichiarata e per questo si possono usare solo i link a disposizione: il puntatore di catena dinamica o il puntatore di catena statica. Abbiamo visto che nel RdA di un blocco non anonimo abbiamo due puntatori. Quale dei due usiamo? Vale prima la pena sottolineare che che lo scope statico o dinamico hanno senso solo in presenza di ambiente non locale e non globale, dato che l'ambiente globale lo possiamo realizzare all'esterno della pila dei RdA. Il *puntatore di catena statica* è quello usato per realizzare lo scope statico (che abbiamo detto è un po' più complesso da implementare dato che dobbiamo *calcolare* questo puntatore), mentre per realizzare lo scope dinamico basta il puntatore di catena dinamica, che sappiamo calcolare facilmente.

Implementazione dello scope statico

Ricordiamo che nello scope *statico* si fa riferimento al legame creato nel blocco testualmente più vicino, ed al momento dell'esecuzione, come si trova il RdA del blocco *testualmente* più vicino? Devo fare attenzione, dato che tra il RdA *attivo* (quello in cima alla pila) e quello che contiene il non locale al quale sono interessato ci possono essere RdA che contengono come nome locale proprio quello. Usiamo l'*annidamento* dei blocchi.

Definizione 15 *Dato un frammento di programma scritto in un linguaggio $\mathcal{L}$ ad alto livello, l'annidamento misura il numero di blocchi/sottoprogrammi aperti e non ancora richiusi all'interno del quale si trova l'occorrenza di un certo nome.*

Ricordiamo che i nomi che usiamo devono essere in *scope*, che significa che devono essere noti.

Per calcolare il puntatore di catena statico abbiamo a disposizione solo il puntatore di catena statico del chiamante, cioè del blocco dove avviene la chiamata alla funzione o da dove si accede ad un nuovo blocco annidato all'interno. Abbiamo due casi:

- il chiamato si trova all'*interno* del chiamante, e qui è tutto facile: il puntatore di catena statica è il puntatore di catena dinamica coincidono, oppure
- il chiamato si trova all'*esterno* del chiamante, quindi dato che il chiamato è un nome noto, lo avremo trovato utilizzando il puntatore di catena statico del chiamante. In che modo? Siamo risaliti lungo la *catena statica* fino a che non abbiamo trovato il nome corretto, e questa *distanza* è nota al momento della scrittura del programma, quindi basta aggiornare il puntatore di catena statica del nuovo RdA in base a questo calcolo.

Il linguaggio FUN con dichiarazioni locali e funzioni

Vediamo prima come aggiungere le dichiarazioni locali, quindi il costrutto che definisce i blocchi in questo linguaggio che è il ‘*let ... in ...*’.

La sintassi diventa

$$t ::= x \mid n \mid \text{true} \mid \text{false} \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid t_1 < t_2 \mid \\ t_1 = t_2 \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid \text{fun}(x, t) \mid \text{apply}(t_1, t_2) \mid \text{let } x \Leftarrow t_1 \text{ in } t_2$$

e assumiamo che se abbiamo $\text{let } x \Leftarrow t_1 \text{ in } t_2$ allora $x \notin FV(t_1)$.

Abbiamo osservato che $\text{let } x \Leftarrow t_1 \text{ in } t_2 \sim \text{apply}(\text{fun}(x, t_2), t_1)$ ma il punto è che ora nei termini t_1 e t_2 possono comparire nomi che sono dichiarati prima. La maggior parte delle regole per ora non variano, ma per completezza le riportiamo tutte. Prima osserviamo che con il nuovo costrutto dobbiamo aggiungere un caso per determinare le free variables del nuovo termine.

$$FV(\text{let } x \Leftarrow t_1 \text{ in } t_2) = (FV(t_2) \setminus \{x\}) \cup FV(t_1)$$

Inoltre assumiamo ci sia un modo per dire, nell’ambiente, quali sono i nomi *validi*, quelli cioè a cui è associato un valore, e questo è $\text{dom}(\delta)$. Questa funzione restituisce il sottoinsieme di nomi a cui è stato associato un valore nel linguaggio. Per ora i valori non sono cambiati.

$$\frac{}{\langle x, \delta \rangle \rightarrow \delta(x)} \quad \frac{}{\langle n, \delta \rangle \rightarrow \mathbf{n}} \quad \frac{}{\langle \text{true}, \delta \rangle \rightarrow \mathbf{true}} \quad \frac{}{\langle \text{false}, \delta \rangle \rightarrow \mathbf{false}} \\ \frac{\langle t, \delta \rangle \rightarrow \mathbf{v}}{\langle \neg t, \delta \rangle \rightarrow \neg \mathbf{v}} \quad \frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2}{\langle t_1 \text{ op } t_2, \delta \rangle \rightarrow \mathbf{v}_1 \text{ op } \mathbf{v}_2} \quad \mathbf{op}, \text{op} \in \{+, -, \times, <, =, \wedge, \vee\}$$

con $\mathbf{v}_1 \text{ op } \mathbf{v}_2$, dove $\mathbf{op} \in \{+, -, \times, <, =, \wedge, \vee\}$ intendiamo che $\mathbf{v}_1$ e $\mathbf{v}_2$ sono definiti e del *tipo* corretto e quindi anche $\mathbf{v}_1 \text{ op } \mathbf{v}_2$ è definito, mentre op è il corrispettivo nel linguaggio di programmazione.

$$\frac{\langle t_0, \delta \rangle \rightarrow \mathbf{true} \quad \langle t_1, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \quad \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{false} \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}}$$

$$\frac{FV(t) \subseteq \{x\} \cup \text{dom}(\delta)}{\langle \text{fun}(x, t), \delta \rangle \rightarrow \text{fun}(x, t)} \quad \frac{\langle t_1, \delta \rangle \rightarrow \text{fun}(x, t) \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2 \quad \langle t, \delta[x/\mathbf{v}_2] \rangle \rightarrow \mathbf{v}}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow \mathbf{v}}$$

$$\frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta[x/\mathbf{v}_1] \rangle \rightarrow \mathbf{v}}{\langle \text{let } x \Leftarrow t_1 \text{ in } t_2, \delta \rangle \rightarrow \mathbf{v}}$$

La domanda a cui vogliamo rispondere è questa: visto che abbiamo le dichiarazioni locali ed i programmi sono strutturati, quale regola di scope abbiamo implementato? Abbiamo implementato lo scope *dinamico*. Infatti la regola per l'*apply* impone che il termine che deve essere un'astrazione funzionale dia un oggetto del tipo corretto (un'astrazione funzionale) ed i nomi non locali vengano risolti nell'ambiente al momento dell'uso del nome.

Scope statico. In questo caso dobbiamo ricordare le associazioni al momento dell'incontro del nome a cui è associata una funzione. Le regole diventano:

$$\begin{array}{c}
\overline{\langle x, \delta \rangle \rightarrow \delta(x)} \qquad \overline{\langle n, \delta \rangle \rightarrow \mathbf{n}} \qquad \overline{\langle \text{true}, \delta \rangle \rightarrow \mathbf{true}} \qquad \overline{\langle \text{false}, \delta \rangle \rightarrow \mathbf{false}} \\
\\
\frac{\langle t, \delta \rangle \rightarrow \mathbf{v}}{\langle \neg t, \delta \rangle \rightarrow \neg \mathbf{v}} \qquad \frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2}{\langle t_1 \text{ op } t_2, \delta \rangle \rightarrow \mathbf{v}_1 \text{ op } \mathbf{v}_2} \quad \text{op, op} \in \{+, -, \times, <, =, \wedge, \vee\} \\
\\
\frac{\langle t_0, \delta \rangle \rightarrow \mathbf{true} \quad \langle t_1, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \qquad \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{false} \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \\
\\
\frac{FV(t) \subseteq \{x\} \cup \text{dom}(\delta)}{\langle \text{fun}(x, t), \delta \rangle \rightarrow (\text{fun}(x, t), \delta)} \quad \frac{\langle t_1, \delta \rangle \rightarrow (\text{fun}(x, t), \delta') \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2 \quad \langle t, \delta'[x/\mathbf{v}_2] \rangle \rightarrow \mathbf{v}}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow \mathbf{v}} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta[x/\mathbf{v}_1] \rangle \rightarrow \mathbf{v}}{\langle \text{let } x \Leftarrow t_1 \text{ in } t_2, \delta \rangle \rightarrow \mathbf{v}}
\end{array}$$

In effetti sono cambiate le regole dell'applicazione e quelle che restituiscono un'astrazione funzionale, che ora è una coppia funzione e ambiente. Cambia quindi $\mathcal{F}$ che non è più solo un oggetto del tipo $\{\text{fun}(x, t) \mid FV(t) \subseteq \{x\}\}$ ma diventa $\mathcal{F} = \{(\text{fun}(x, t), \delta) \mid FV(t) \subseteq \{x\} \cup \text{dom}(\delta)\}$. Non possiamo però avere ricorsione. Vediamo perché. Prendiamo il termine

$$\text{let } f \Leftarrow \text{fun}(x, \text{apply}(f, x)) \text{ in } \text{apply}(f, 1)$$

che valutiamo nell'ambiente δ tale che $\text{dom}(\delta) = \emptyset$. Dobbiamo applicare la regola per il *let* e le premesse sono che dobbiamo dire quale è il valore associato al termine $\text{fun}(x, \text{apply}(f, x))$ nell'ambiente δ . Dobbiamo quindi applicare la regola

$$\frac{FV(t) \subseteq \{x\} \cup \text{dom}(\delta)}{\langle \text{fun}(x, t), \delta \rangle \rightarrow (\text{fun}(x, t), \delta)}$$

ma $FV(\text{fun}(x, \text{apply}(f, x))) = \{f\}$ che non è contenuto in $\text{dom}(\delta)$ che è vuoto. Quindi la regola non può essere applicata e di conseguenza non siamo in grado di valutare tale termine, sostanzialmente perché il nome f non è noto nell'ambiente δ . Per risolvere questo problema bisogna sostanzialmente cambiare un po' il linguaggio aggiungendo alla sintassi le funzioni (potenzialmente) ricorsive. La sintassi quindi diventa

$$\begin{aligned}
t ::= & x \mid n \mid \text{true} \mid \text{false} \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid t_1 < t_2 \mid \\
& t_1 = t_2 \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid \text{fun}(x, t) \mid \text{apply}(t_1, t_2) \mid \text{let } x \Leftarrow t_1 \text{ in } t_2 \mid \\
& \text{rec}(y, \text{fun}(x, t))
\end{aligned}$$

mentre le regole della semantica sono le seguenti:

$$\begin{array}{c}
\frac{}{\langle x, \delta \rangle \rightarrow \delta(x)} \quad \frac{}{\langle n, \delta \rangle \rightarrow \mathbf{n}} \quad \frac{}{\langle \text{true}, \delta \rangle \rightarrow \mathbf{true}} \quad \frac{}{\langle \text{false}, \delta \rangle \rightarrow \mathbf{false}} \\
\\
\frac{\langle t, \delta \rangle \rightarrow \mathbf{v}}{\langle \neg t, \delta \rangle \rightarrow \neg \mathbf{v}} \quad \frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2}{\langle t_1 \text{ op } t_2, \delta \rangle \rightarrow \mathbf{v}_1 \text{ op } \mathbf{v}_2} \quad \text{op, op} \in \{+, -, \times, <, =, \wedge, \vee\} \\
\\
\frac{\langle t_0, \delta \rangle \rightarrow \mathbf{true} \quad \langle t_1, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \quad \frac{\langle t_0, \delta \rangle \rightarrow \mathbf{false} \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}}{\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, \delta \rangle \rightarrow \mathbf{v}} \\
\\
\frac{FV(t) \subseteq \{x\} \cup \text{dom}(\delta)}{\langle \text{fun}(x, t), \delta \rangle \rightarrow (\text{fun}(x, t), \delta)} \quad \frac{\langle t_1, \delta \rangle \rightarrow (\text{fun}(x, t), \delta') \quad \langle t_2, \delta \rangle \rightarrow \mathbf{v}_2 \quad \langle t, \delta'[x/\mathbf{v}_2] \rangle \rightarrow \mathbf{v}}{\langle \text{apply}(t_1, t_2), \delta \rangle \rightarrow \mathbf{v}} \\
\\
\frac{\langle t_1, \delta \rangle \rightarrow \mathbf{v}_1 \quad \langle t_2, \delta[x/\mathbf{v}_1] \rangle \rightarrow \mathbf{v}}{\langle \text{let } x \Leftarrow t_1 \text{ in } t_2, \delta \rangle \rightarrow \mathbf{v}} \quad \frac{\delta' = \delta[y/(\text{fun}(x, t), \delta')]}{\langle \text{rec}(y, \text{fun}(x, t)), \delta \rangle \rightarrow (\text{fun}(x, t), \delta')}
\end{array}$$

Il problema ora è capire come costruiamo l'ambiente per $\text{rec}(y, \text{fun}(x, t))$. Abbiamo bisogno di un po' di teoria. Dato che dobbiamo *costruire* il più piccolo ambiente δ' che contiene l'associazione tra il nome della funzione f e il suo codice $(\text{fun}(x, t))$ e l'ambiente δ' medesimo, dobbiamo definire un'opportuna funzione $F : Env \rightarrow Env$ che dovrà essere tale che, per quel particolare δ che cerchiamo, soddisfi la seguente equazione: $F(\delta) = \delta$. La prima definizione che ci serve è quella di *complete partial order*. Per poterlo definire abbiamo bisogno delle nozioni di ordine parziale, di ω -chain, di maggiorante e minorante di un insieme.

Definizione 16 Un ordine parziale è una coppia $(D, \sqsubseteq)$ dove D è un insieme e $\sqsubseteq \subseteq D \times D$ è una relazione di ordine parziale, che quindi è riflessiva, antisimmetrica e transitiva. L'insieme D è chiamato supporto.

Una ω -chain di elementi in D è una sequenza $d_1, \dots, d_n, \dots$ potenzialmente infinita tale che $\forall i \in \omega$ vale che $d_i \sqsubseteq d_{i+1}$.

Dato un sottoinsieme $D' \subseteq D$ di elementi di D , diremo che d è un maggiorante di D' se vale che per ogni $d' \in D$. $d' \sqsubseteq d$. Dato $D' \subseteq D$, un maggiorante d è il minimo dei maggioranti se vale che è un maggiorante ed ogni altro maggiorante di D' è maggiore o uguale a d .

Analogamente si definiscono i minoranti ed i massimi dei minoranti.

Facciamo alcuni esempi di ordini parziali, aderenti a cosa abbiamo visto. L'insieme dei numeri interi $\mathbb{Z}$ con come ordine parziale è quello definito come $n \sqsubseteq m$ se e solo se $n = m$

è un ordine parziale, i booleani con come ordine l'identità è un ordine parziale. Anche l'unione di $\mathbb{Z}$ e $\mathbb{B}$ è un ordine parziale, con come ordini l'unione dei due ordini.

Possiamo ora dire cosa è un CPO o complete partial order.

Definizione 17 *Un CPO è un ordine parziale $(D, \sqsubseteq)$ dove ogni ω -chain di elementi in D , vista come sottoinsieme, ha un unico minimo dei maggioranti che è chiamato least upper bound e che viene indicato come $\bigsqcup_{i \in \omega} d_i$.*

Una funzione da un CPO $(D, \sqsubseteq)$ ad un CPO $(D', \sqsubseteq')$ è una funzione da D a D' , quindi una funzione tra i supporti. Tale funzione è *monotona* se $d \sqsubseteq d'$ implica che $f(d) \sqsubseteq' f(d')$. Tale funzione è detta anche continua se, per ogni ω -chain di elementi in D , vale che $f(\bigsqcup_{i \in \omega} d_i) = \bigsqcup_{i \in \omega} f(d_i)$. Osserviamo che il nostro ambiente è una funzione continua.

L'ultimo tassello che ci manca è studiare come è fatto Env . Definiamo un ordine sugli ambienti. Poniamo $\delta \sqsubseteq \delta'$ se e solo se per ogni $x \in Var$. $\delta(x) \downarrow \Rightarrow \delta'(x) \downarrow$ e $\delta(x) = \delta'(x)$.

Teorema 1 $(Env, \sqsubseteq)$ è un CPO.

La trasformazione sugli ambienti indotta dalla regola

$$\frac{\delta' = \delta[y/(fun(x, t), \delta')]}{\langle rec(y, fun(x, t)), \delta \rangle \rightarrow (fun(x, t), \delta')}$$

che per convenienza chiamiamo $F : Env \rightarrow Env$ è una funzione continua ed è facile convincersi che la sequenza di ambienti che otteniamo è proprio una ω -chain, quindi ne possiamo costruire effettivamente il least upper bound. Infatti abbiamo che otteniamo δ dove $\delta(y) = (fun(x, t), \delta)$ e se lo *aggiorniamo* usando la regola per le astrazioni funzionali e per la ricorsione abbiamo che ad y associamo propriamente $(fun(x, t), \delta)$.

Il linguaggio IMP con i blocchi e le dichiarazioni (locali)

Arricchiamo il linguaggio imperativo IMP con i blocchi e le dichiarazioni locali.

Dichiarazioni: Le dichiarazioni sono le seguenti

$$dich ::= \text{null} \mid \text{dseq}(dich_1, dich_2) \mid \text{Var}(X)$$

null è la dichiarazione vuota.

Lo stato ora sarà composto da un'associazione nomi - oggetti denotabili (che contengono anche le locazioni), che chiameremo ambiente, e da una *memoria* che associa alle locazioni gli oggetti memorizzabili. La semantica delle dichiarazioni è:

$$\begin{array}{c} \frac{}{\langle \text{null}, (\rho, \sigma) \rangle \mapsto_d (\rho, \sigma)} \quad \frac{loc = \text{newloc}()}{\langle \text{Var}(X), (\rho, \sigma) \rangle \mapsto_d (\rho[X/loc], \sigma)} \\[10pt] \frac{\langle d_1, (\rho, \sigma) \rangle \mapsto_d (\rho'', \sigma'') \quad \langle d_2, (\rho'', \sigma'') \rangle \mapsto_d (\rho', \sigma')}{\langle \text{dseq}(d_1, d_2), (\rho, \sigma) \rangle \mapsto_d (\rho', \sigma')} \end{array}$$

dove $\text{newloc}()$ è una funzione che restituisce una nuova locazione.

Comandi: Ai comandi aggiungiamo **block**(*dich*, *com*), ma avendo modificato lievemente la nozione di stato, riscriviamo tutte le regole.

$$\begin{array}{c}
\frac{}{\langle \text{skip}, (\rho, \sigma) \rangle \mapsto_c \sigma} \qquad \frac{\langle e, (\rho, \sigma) \rangle \mapsto_v \mathbf{v}}{\langle \text{assign}(X, e), (\rho, \sigma) \rangle \mapsto_c \sigma[\rho(X)/\mathbf{v}]} \\
\\
\frac{\langle c_1, (\rho, \sigma) \rangle \mapsto_c \sigma'' \quad \langle c_2, (\rho, \sigma'') \rangle \mapsto_c \sigma'}{\langle \text{cseq}(c_1, c_2), (\rho, \sigma) \rangle \mapsto_c \sigma'} \\
\\
\frac{\langle b, (\rho, \sigma) \rangle \mapsto_v \mathbf{true} \quad \langle c_1, (\rho, \sigma) \rangle \mapsto_c \sigma'}{\langle \text{if}(b, c_1, c_2), (\rho, \sigma) \rangle \mapsto_c \sigma'} \quad \frac{\langle b, (\rho, \sigma) \rangle \mapsto_v \mathbf{false} \quad \langle c_2, (\rho, \sigma) \rangle \mapsto_c \sigma'}{\langle \text{if}(b, c_1, c_2), (\rho, \sigma) \rangle \mapsto_c \sigma'} \\
\\
\frac{\langle b, (\rho, \sigma) \rangle \mapsto_v \mathbf{false}}{\langle \text{while}(b, c), (\rho, \sigma) \rangle \mapsto_c \sigma} \\
\\
\frac{\langle b, (\rho, \sigma) \rangle \mapsto_v \mathbf{true} \quad \langle c, (\rho, \sigma) \rangle \mapsto_c \sigma'' \quad \langle \text{while}(b, c), (\rho, \sigma'') \rangle \mapsto_c \sigma'}{\langle \text{while}(b, c), (\rho, \sigma) \rangle \mapsto_c \sigma'}
\end{array}$$

mentre quella per i blocchi sarà:

$$\frac{\langle d, (\rho, \sigma) \rangle \mapsto_d (\rho'', \sigma'') \quad \langle c, (\rho'', \sigma'') \rangle \mapsto_c \sigma'}{\langle \text{block}(d, c), (\rho, \sigma) \rangle \mapsto_c \sigma'}$$

Parametri, astrazione e higher order

Programmi strutturati

Abbiamo visto che i programmi di un linguaggio di programmazione sono in generale *strutturati*. Essere strutturati ha una importante conseguenza: gli elementi per capire cosa deve essere eseguito sono tutti contenuti nella porzione di programma che si sta eseguendo. Nel caso di un programma di un linguaggio appartenente al paradigma funzionale abbiamo che l'applicazione determina cosa dobbiamo eseguire, nel caso della programmazione imperativa e dichiarativa la istruzione successiva da eseguire è scritta nell'istruzione medesima che viene eseguita, mentre nel paradigma orientato ad oggetti i messaggi sono tra oggetti in quel momento attivi.

Abbiamo visto che il modo di strutturare i programmi è attraverso i blocchi, che possono essere o anonimi oppure avere un nome, e questi costituiscono sostanzialmente le funzioni, procedure e moduli che usiamo per strutturare i programmi. Un blocco, che abbia un nome o meno, è in genere costituito da due parti: una dichiarazione, che ci serve per costruire l'ambiente δ , e un corpo che va eseguito tenendo conto dell'ambiente costruito dalla dichiarazione.

I vantaggi di questo approccio sono:

- facilita la progettazione top-down (gerarchica) dei programmi,
- consente la *modularizzazione* del codice, che quindi può essere diviso,
- consente l'uso di nomi significativi per denotare oggetti,
- consente l'uso di tipi di dato strutturati e di costrutti strutturati per il controllo come l'*if* oppure, nel caso del paradigma imperativo, il *while*,
- con i commenti rende il codice comprensibile.

Per poter parlare d'astrazione bisogna anche parlare di *ricorsione*. Per questo ricordiamo che le *definizioni induttive* sono quelle date dicendo prima cosa si fa nei casi base e poi cosa si fa in nei casi *generici*. L'esempio classico, il solito fattoriale:

$$fatt(x) = \begin{cases} 1 & \text{se } x = 0 \\ x \times fatt(x - 1) & \text{altrimenti} \end{cases}$$

Queste definizioni le posso vedere come delle *dichiarazioni* su cosa devo fare per calcolare (da qui i linguaggi del paradigma dichiarativo).

Le funzioni ricorsive richiamano se stesse, e il *calcolo* viene eseguito usando la pila dei RdA. Abbiamo un caso particolare che bisogna ricordare: la ricorsione in coda (tail-recursion).

Definizione 18 *Sia f una funzione che nel suo corpo contiene una chiamata alla funzione g (non necessariamente diversa da f). La chiamata di g si dice chiamata in coda se la funzione f restituisce il valore restituito da g senza dover fare alcuna ulteriore computazione.*

Diciamo che f è tail-recursive (ha la ricorsione in coda) se tutte le chiamate ricorsive in f sono chiamate in coda.

Se la chiamata ricorsiva è l'ultima istruzione allora non mi devo ricordare i punti di ritorno e mi basta mettere sulla pila un solo RdA e non tutti quelli delle istanze di chiamata della funzione. Con un po' di trasformazioni molte funzioni possono essere rese tail-recursive.

Osserviamo che non c'è alcuna sostanziale differenza tra ricorsione ed iterazione dal punto di vista teorico. Un programma iterativo può essere reso ricorsivo (l'idea è che un *while* lo vedo come una funzione a cui passo come parametro la condizione booleana) ed un programma ricorsivo può essere reso iterativo nello stesso modo. Un tempo l'iterazione era considerata più efficiente, dato che un compilatore la *traduceva* in codice *macchina* facilmente, mentre la ricorsione era considerata più *elegante* ma meno efficiente dato che si calcola con la pila dei record d'attivazione. Ora questa differenza è considerata risibile, quindi possiamo dire che non c'è reale differenza.

Astrazione

Abbiamo detto che l'astrazione è quel meccanismo che consente *realmente* di avere linguaggi ad alto livello. Con l'astrazione possiamo identificare proprietà importanti di cosa si vuole descrivere, e possiamo focalizzare l'attenzione sulle questioni rilevanti, ignorando quelle meno importanti, infine consente di specificare cosa interessa, facilitando in questo modo la *scrittura* del programma. Ovviamente cosa è rilevante dipende da cosa si vuole ottenere.

Nel caso del paradigma imperativo, astrarre il controllo significa nascondere dettagli procedurali.

Vediamo ora come si realizza effettivamente l'astrazione.

Sottoprogrammi e parametri

Un *sottoprogramma* è una parte di programma che ha un *nome*, ha una sequenza di *parametri* (che vengono chiamati parametri *formali* nella definizione del sottoprogramma e *attuali* quando il sottoprogramma viene effettivamente usato) e può restituire un *valore* (quindi essere una funzione ed avere un tipo) o un oggetto. Se non restituisce alcunché, può modificare lo stato della macchina astratta. In generale nel linguaggio c'è la possibilità di *definirli* e di *invocarli*.

Ricordiamo che storicamente si usava la keyword **function** per le astrazioni che restituiscono un valore e la keyword **procedure** per quelle che non restituiscono un valore. Nei linguaggi moderni sono tutte funzioni, e le procedure restituiscono l'unico elemento del tipo **void**.

Perché introdurre l'astrazione? In generale un frammento di programma (sequenza di istruzioni) risulta utile in diversi punti del programma, e si riduce il “costo della programmazione” se si può dare un nome al frammento e qualcun altro inserisce automaticamente il codice del frammento ogni qualvolta nel “programma principale” c'è un'occorrenza del nome (attenzione che l'astrazione è diversa da avere le *macro* e la conseguente macro-espansione, che è una sostituzione testuale. Si riduce anche l'occupazione di memoria se esiste un meccanismo che permette al programma principale di trasferire il *controllo* ad una unica copia del frammento memorizzata separatamente e di riprendere il controllo quando l'esecuzione del frammento termina (ricordiamo che la subroutine è in generale supportata anche dall'hardware (codice rientrante)). Infine il frammento diventa ancora più importante se può essere realizzato in modo parametrico.

Vediamo come funziona in generale. A livello hardware, c'è una operazione primitiva di **return jump**. Quando viene eseguita (nel programma chiamante) l'istruzione **return jump** *a* memorizzata nella cella *b* il controllo viene trasferito alla cella *a* (entry point della subroutine) e l'indirizzo dell'istruzione successiva *b + 1* viene memorizzato da qualche parte prefissata (punto di ritorno). Quando nella subroutine si esegue una operazione di **return** il controllo *ritorna* all'istruzione (del programma chiamante) memorizzata nel punto di ritorno.

Nel Fortran (imperativo) una subroutine è (meglio era) un pezzo di codice compilato, a cui sono associati una cella destinata a contenere (a tempo di esecuzione) i punti di

ritorno relativi alle (possibili varie) chiamate e alcune celle destinate a contenere i valori degli eventuali parametri, oltre all'ambiente e memoria locali (in Fortran l'ambiente locale è statico).

Nel Fortran la semantica delle subroutine si definisce facilmente attraverso la copy rule statica, cioè la macroespansione. Ogni chiamata di sottoprogramma è testualmente rimpiazzata da una copia del codice facendo qualcosa per i parametri e ricordandosi che le dichiarazioni sono eseguite una sola volta.

Il sottoprogramma non è *semanticamente* qualcosa di nuovo: è solo un (importante) strumento metodologico e non è compatibile con la ricorsione. Il fatto che le subroutine Fortran siano concettualmente una cosa statica fa sì che non esista di fatto il concetto di attivazione e l'ambiente locale è necessariamente statico.

Noi vogliamo una vera nozione di sottoprogramma e poter ragionare in termini di attivazioni. La semantica può essere ancora definita da una copy rule, ma deve essere dinamica, cioè ogni chiamata di sottoprogramma è rimpiazzata a tempo di esecuzione da una copia del codice e ragionare in termini di *attivazione* ci dà un sottoprogramma che è ora *semanticamente* qualcosa di nuovo, rende naturale la ricorsione e porta ad adottare la regola dell'ambiente locale dinamico.

Con l'*astrazione procedurale* si realizza sia l'astrazione di una sequenza di istruzioni che l'astrazione via parametrizzazione. Cosa serve per realizzarla? Un luogo di controllo per la gestione dell'ambiente e della memoria (pila degli RdA), ed è un'estensione della nozione di blocco (senza funzioni o procedure è staticamente chiaro cosa c'è), ed è in assoluto l'aspetto più interessante dei linguaggi, intorno a cui ruotano tutte le decisioni semantiche importanti.

Invece delle informazioni (punto di ritorno, parametri, ambiente e memoria locale) staticamente associate al codice compilato del Fortran vogliamo avere i record di attivazione, che contengono più o meno le stesse informazioni, e questi devono essere associati dinamicamente alle varie chiamate di sottoprogrammi. Ci possiamo aspettare che i record di attivazione siano organizzati in una pila dato che i sottoprogrammi hanno un comportamento LIFO e l'ultima attivazione creata nel tempo è la prima che *ritorna*. Ecco perché la pila degli RdA è così importante.

C'è flusso d'informazione tra programma e sottoprogramma: quando eseguiamo l'invocazione di un sottoprogramma, questo "comunica" con il chiamante attraverso i seguenti mezzi

- l'ambiente non locale (il meccanismo meno sofisticato),
- i parametri, che sono passati in vari *modi*: per *valore*, per *referimento*, per *costante*, per *risultato*, per *valore-risultato* e per *nome*,
- il valore restituito se è una funzione.

L'invocazione corrisponde a mettere sulla pila degli RdA il record d'attivazione del sottoprogramma e quindi calcolare lo stato della macchina astratta in cui eseguire le istruzioni

del sottoprogramma, cioè l'ambiente, la memoria ed eventuali strutture dinamiche come lo heap.

Vediamo i vari modi di passaggio dei parametri:

Passaggio per valore

Comunicazione unidirezionale: dal programma al sottoprogramma. È spesso la modalità di default per cui non ci sono particolari keywords (le mettiamo solo se vogliamo sottolineare la modalità). Il **valore** del parametro attuale (che deve quindi essere di un tipo che ha valori, non `void` che ha un solo valore) viene associato al parametro formale. Nei linguaggi funzionali l'operazione è sull'ambiente mentre nei linguaggi imperativi l'operazione è sull'ambiente e sulla memoria.

Non c'è comunicazione tra chiamante e chiamato attraverso il parametro, dato che non c'è alcun legame tra il parametro formale e quello attuale. È costoso per dati *grandi*: se ne creano copie.

In Java, Scheme, Pascal è il modo di default, per C e Java è anche l'unico modo.

Passaggio per riferimento

Comunicazione bidirezionale tra programma e sottoprogramma, tipica dei linguaggi imperativi.

Si associa al parametro formale il *riferimento* al parametro attuale (che non viene valutato). Il parametro attuale deve essere una variabile a cui è associata una locazione e l'operazione è sempre sull'ambiente. Il parametro formale è un'*alias* del parametro attuale, ed è efficiente dato che non si valuta e si cambia solo l'ambiente. Gli effetti si ripercuotono all'esterno, quindi attenzione.

Nel Pascal (keyword `var`), in C se passo un puntatore ho lo stesso effetto (quindi è simulabile), e per questo il C ha solo il passaggio per valore. Quello per riferimento è simulato.

Passaggio per costante

Comunicazione unidirezionale e obsoleta: da programma a sottoprogramma. Associa al parametro formale la *valutazione* del parametro attuale (come nel passaggio per valore) ma nel sottoprogramma non è consentito modificare il parametro formale. Implementazione a discrezione (per valore nel caso di dati di piccole dimensioni, per riferimento in altri casi), ma il reale controllo è fatto sull'uso del parametro attuale. Presente in Java (keyword `final`), Modula-3, ANSI C (keyword `const`)

Passaggio per risultato

Comunicazione unidirezionale: da sottoprogramma a programma. Associa al parametro attuale la *valutazione* del parametro formale all'*uscita* dal sottoprogramma. Il parametro formale non viene manipolato in ingresso ma in uscita, e il parametro attuale deve essere un nome associato ad una locazione. Nel sottoprogramma non c'è alcun legame tra

parametro formale e parametro attuale. Tale metodo è costoso per dati “grandi” (è sempre una copia). Come per costante. Presente in Java (keyword `final`), Modula-3, ANSI C (keyword `const`)

Passaggio per valore-risultato

Comunicazione bidirezionale tra programma e sottoprogramma, tipica dei linguaggi imperativi in cui si vogliono restituire dati strutturati.

In ingresso associo al parametro formale il *valore* del parametro attuale, mentre in uscita faccio il viceversa, associo al parametro attuale il *valore* del parametro formale. Il parametro attuale deve essere un nome associato ad una locazione e l’operazione è sempre sugli ambienti (del chiamato e del chiamante).

Il parametro formale in questo caso *non* è un *alias* del parametro attuale.

Passaggio per nome

Metodo di comunicazione bidirezionale tra programma e sottoprogramma. Nasce con Algol-W (era il modo), ed è una sorta di sostituzione testuale: nel sottoprogramma le occorrenze del parametro formale vengono sostituite con il parametro attuale (che deve essere una variabile modificabile, quindi un nome a cui è associata una locazione). Considerate il programma

```
1. int y;  
2. void passnome (name int x) {  
    x = x + 1; }  
    ...  
n. y = 1;  
    ...  
m. passnome(y)
```

In questo frammento abbiamo che, al momento della chiamata della funzione `passnome` *nome* del parametro attuale `y` viene sostituito nel corpo di `passnome` alla linea `m`. Si esegue quindi ‘`y = y + 1`’. Ora il passaggio per nome ha dei problemi. Considerate il frammento

```
1. int y;  
2. void passnome (name int x) {  
3.   int y = 1;  
    x = x + y; }  
    ...  
n. y = 2;  
    ...  
m. passnome(y)
```

se come prima il nome del parametro attuale `y` viene sostituito nel corpo di `passnome` alla linea `m`. Si esegue ‘`y = y + y`’ ma il risultato dipende dal contesto di chiamata. Le

due variabili (y e y) dovrebbero essere diverse, ma come fare per distinguerle? Viene introdotto il concetto di *chiusura*, cioè non si passa solo il nome, ma anche il suo ambiente.

La chiusura è formalmente una coppia $\langle e, \delta \rangle$ dove δ è l'ambiente dove valutare l'espressione e .

```
1. int y;  
2. void passnome (name int x) {  
3.   int y = 1;  
    x = x + y; }  
    ...  
n. y = 2;  
    ...  
m. passnome(y)
```

come prima il nome del parametro attuale y viene sostituito nel corpo di `passnome` alla linea m. Si esegue ' $y = y + y$ ' ma il risultato non dipende dal contesto di chiamata. Le due variabili (y e y) sono diverse, una è quella nella linea 1, l'altra nella linea 3, e ogni variabile viene valutata nel suo ambiente (tramite la chiusura), quindi si crea il corretto aliasing tra formale e attuale.

Per implementare il passaggio per nome, e quindi passare la coppia $\langle e, \delta \rangle$ dove δ è l'ambiente dove valutare l'espressione e , bisogna passare un *puntatore* al testo di e (quindi creare un'associazione con il testo di e) e passare il puntatore di catena statica al RdA del chiamante.

Viene chiamata chiusura perché chiude una espressione con il suo ambiente. Osservate che le variabili *libere* vengono legate, quindi è come definire una funzione. Le chiusure servono a passare funzioni come argomenti ad altre funzioni, oltre che a capire come implementare lo scope statico.

Higher order

Abbiamo visto che si possono senza problemi passare senza problemi oggetti di qualsiasi tipo, spesso e volentieri passando il modo per arrivare all'oggetto. Viene quindi naturale cosa succede se passiamo una funzione come argomento.

Consideriamo il programma

```
1. int z = 0;  
2. int f (int y){  
3.   return y+1;}  
4. int g (int -> int h) {  
5.   return h(3);  
6.   }  
    ...
```

```
m. g(f);
```

qui la funzione `f`, che ha *tipo* `int -> int`, viene passata come argomento alla `g` che la usa nel suo *corpo*. Se è anche vero che non abbiamo mai formalmente definito quale è il tipo di un programma, e quindi di una funzione, abbiamo detto che un programma riceve dei dati in ingresso e restituisce dei dati in uscita, quindi intuitivamente il tipo di uno specifico programma è determinato dal tipo dei dati in ingresso e dal tipo dei dati che restituisce. La `f` riceve un intero e restituisce un intero, mentre la `g` riceve una funzione da interi ad interi e restituisce un intero. Quindi la `g` ha tipo `(int -> int) -> int`.

Cosa succede quando chiamo la `g` e passo come argomento la `f`? Alla linea 5 viene eseguita una chiamata al parametro `h`, a cui è associata la `f`, quindi si chiama la `f`. In questo caso non ci sono variabili non locali quindi non ci sono problemi con lo scope statico o dinamico, ma in generale non è così. L'esecuzione della chiamata `h(3)` fa sì che venga restituita dalla `f` il valore 4 e anche dalla `g`.

Consideriamo il programma

```
1. {int x = 4;
2.  int z = 0;
3.  int f (int y){
4.      return y+x;}
5.  void g (int -> int h) {
6.      int x = 7;
7.      z = h(3) + x;}
..  ...
m.  {int x = 5;
    ..
1.  g(f);
..  ...}
..  ...
.. }
```

Cosa succede quando chiamo la `g` e passo come argomento la `f` che viene eseguita alla linea 8? A quale definizione della `x` ci si riferisce nell'uso della `f`? Ce ne sono 3. Una è quella alla linea 1, nel blocco più esterno, poi ne troviamo una alla linea 6, locale alla `g`, ed infine una locale al blocco che inizia alla linea `m` e all'interno del quale si trova la chiamata alla `g`.

Devo stabilire quali sono le *politiche di binding*. Quando una procedura viene passata come parametro, si crea un riferimento tra un nome, il parametro formale, nel nostro esempio `h`, ed una procedura, nel nostro esempio il parametro attuale `f`. Indichiamo questo legame con `h<->f`. Quale ambiente non locale si applica al momento dell'esecuzione di `f` che è stata chiamata via `h` all'interno della `g`? Le politiche di binding sono

due: *deep binding*: ambiente al momento della creazione del legame $h \leftrightarrow f$, e *shallow binding*: ambiente al momento della chiamata di f via h .

Domandiamoci quale è la relazione con lo scope. Se lo scope è statico, shallow e deep binding non contano, dato che la regola dello scope ci dice quali sono le variabili non locali della procedura passata come parametro (passeremo sempre una chiusura). Se lo scope è dinamico allora la situazione si fa più complessa. Nel caso in cui si implementa il deep binding ci si immagina che il programmatore voglia che l'ambiente sia quello al momento della chiamata della funzione a cui si passa la funzione come parametro, e si realizza un po' di scope statico, mentre se si implementa lo shallow binding vale la solita regola dello scope statico.

Nel concreto, nel nostro esempio, se lo scope è statico la chiamata $h(3)$ fa sì che la x all'interno della f sia quella dichiarata alla linea 1, se lo scope è dinamico e la politica di binding è quella del shallow binding, si applica la regola dello scope dinamico tout court, quindi la x all'interno della f è quella dichiarata alla linea 6, nell'ambiente attivo al momento della invocazione della f , infine se lo scope è dinamico e la politica di binding è quella del deep binding, si applica solo qui limitatamente la regola dello scope statico, quindi la x all'interno della f è quella dichiarata alla linea m , cioè quella nell'ambiente attivo al momento della invocazione della g e quindi della creazione del legame $h \leftrightarrow f$.

Nel caso dello scope statico, per passare dinamicamente sia il legame al codice della funzione che il suo ambiente non locale, passiamo una chiusura $\langle \text{code}, \delta \rangle$. Alla chiamata di una procedura passata per parametro, si alloca (come sempre) il record di attivazione sulla pila e si prende il puntatore di catena statica dalla chiusura (della funzione), quindi le chiusure servono per mantenere puntatore all'ambiente statico del corpo di una funzione. Tutti i puntatori di catena statica puntano sempre all'*indietro* nella pila, quindi alcuni RdA vengono *saltati* per accedere alle variabili non locali e la de-allocazione dei record di attivazione avviene sempre secondo la politica LIFO (abbiamo sempre la pila). Per implementare lo scope dinamico con la politica del deep binding devo usare le chiusure per *congelare* uno scope da riattivare dopo, mentre per lo shallow binding non devo fare nulla di particolare.

Consideriamo il frammento

```
void A (void -> int f, int x) {
    int B () {
        return x;}
    int z;
    if x==0 then {z = f();}
        else {A(B,0);}
}
...
int C () {
    return 1;}
...
A(C,1);
```

a quale x si fa riferimento se chiamo $A(B, 0)$ in A (e quindi chiamerò eventualmente la B)? Lo scope da solo non basta. Se la politica è quella dello *shallow* binding quello al momento dell'uso, quindi a x è associato il valore 0, mentre se la politica è quella del *deep* binding quello al momento della creazione del legame tra la f e la B , quindi la x contiene 1.

Abbiamo visto come passare le funzioni come parametri, vediamo cosa succede se vogliamo restituire una funzione come risultato. In molti linguaggi possiamo definire tipi complessi (ad esempio, una funzione) ed è quindi naturale immaginare di poter scrivere sottoprogrammi che definiscono funzioni. Abbiamo la creazione *dinamica* di funzioni al momento dell'esecuzione.

Ricordiamo che quando eseguiamo una funzione dobbiamo conoscere l'ambiente non locale della funzione, quindi questo come lo determino?

consideriamo

```
1.  int x = 1;
2.  void -> int F () {
3.      int g () {
4.          return x + 1;
5.      }
6.      return g;
7.  }
...
m.  void -> int gg = F();;
...
1.  int z = gg();
```

La F restituisce una funzione che ha gli stessi riferimenti non locali di dove viene *creata* ed un suo ambiente, quindi restituisco sempre una chiusura.

Il tipo del valore restituito da F è `void -> int` dato che nel corpo della F c'è la definizione della funzione che dobbiamo restituire, e quando si invoca la F (alla linea m) si dà un nome alla funzione restituita (gg) che viene *usata* alla linea l . L'effetto è di incrementare x della linea 1.

Non ci sono problemi perché, anche se il RdA di F non è più sulla pila, c'è quello che ci interessa: il riferimento non locale x . Sostanzialmente è per questo bisogna restituire una chiusura.

Ma consideriamo

```
1.  void -> int F () {
2.      int x = 1;
3.      int g () {
4.          return x + 1;
5.      }
6.      return g;
7.  }
```



```

...
m. void -> int gg = F();
...
l. int z = gg();

```

Quando si invoca la *F* (linea *m*) si dà un nome alla funzione restituita (*gg*) che viene *usata* alla linea *l*. L'effetto è di incrementare *x* di 1. Ora ci sono problemi perché il RdA di *F* non è più sulla pila e quindi *non* c'è quello che interessa: il riferimento non locale a *x*, e questo anche se si restituisce una chiusura, che contiene un puntatore ad una parte che non esiste più.

Bisogna *derogare* alla politica FIFO per gli RdA e *conservare* quell'ambiente. Bisogna quindi non deallocare gli RdA esplicitamente, ma metterli sullo heap e collegarli con i vari puntatori, indicare quando non sono *in esecuzione* ma il loro ambiente è attivo. La soluzione adottata è che di solito la pila degli RdA viene allocata sullo heap.

Dati e Tipi di Dato

Cosa è un *dato*? In generale è un oggetto appartenente ad un qualche *tipo* che viene manipolato dal programma. Possiamo quindi dare una definizione di *tipo di dato*.

Definizione 19 *Un tipo di dato è una collezione di valori omogenei ed effettivamente presentati, dotata di un insieme di operazioni che manipolano tali valori.*

Abbiamo che i valori di un tipo di dato devono essere *omogenei*, cioè devono avere le stesse caratteristiche), e devono essere *effettivamente* presentati, quindi abbiamo per ogni valore una rappresentazione effettiva.

Non deve sfuggire il fatto che la definizione implica che i valori sono un *insieme*, che questo insieme è dotato di operazioni (un'algebra) e la richiesta che siano effettivamente presentati implica che li sappiamo manipolare in una macchina astratta. Bisogna fare *attenzione*: cosa è un tipo e cosa invece non lo è dipende *fortemente* dal linguaggio di programmazione

A che servono i tipi di dato? Nel progetto della soluzione di un problema consentono di organizzare l'informazione:

- tipi diversi per concetti diversi
- un identificatore deve venir usato con il tipo corretto: il tipo è una sorta di *commento* sull'uso dell'identificatore

mentre al momento della scrittura del programma consentono di identificare e prevenire errori

- i tipi sono controllabili automaticamente, e sono un controllo *dimensionale* (3+"pippo" deve essere sbagliato)

Infine, al momento dell'implementazione del linguaggio permettono alcune ottimizzazioni

- `bool` richiede meno bit di `real`
- precalcolo di offset per l'accesso a `struct`

Sistema di tipi

Definizione 20 *Un sistema di tipi è il complesso delle informazioni e delle regole che governano i tipi di un linguaggio di programmazione.*

Nel sistema di tipi ci saranno i tipi predefiniti (spesso detti di base), i meccanismi per definire nuovi tipi e dei meccanismi relativi al controllo dei tipi. In particolare meccanismi per dire se due tipi sono equivalenti (quali sono formalmente diversi ma sono poi lo stesso tipo), quali sono compatibili (quando un valore di un tipo può essere usato come se fosse di un altro tipo) e come fare per attribuire un tipo ad un'espressione complessa (inferenza di tipo). Inoltre il sistema di tipi specifica se i controlli (e anche quali controlli) sono da effettuare staticamente (prima dell'esecuzione) o dinamicamente (al momento dell'esecuzione).

Type safeness

Definizione 21 *Un sistema di tipi è type safe se in esecuzione non possono avvenire errori non segnalati derivanti da un errore di tipo.*

Un linguaggio type safe viene detto anche fortemente tipizzato.

Tipi di dato base

I valori di un tipo possono essere

- *denotabili*: possono essere associati ad un nome
- *esprimibili*: possono essere il risultato della valutazione di un'espressione complessa
- *memorizzabili*; possono essere associati ad una locazione

Ovviamente le scelte di cosa è denotabile, memorizzabile, esprimibile dipendono dal linguaggio (e quindi dalla sua progettazione e implementazione)

I controlli di tipo possono essere statici, fatti cioè al momento della compilazione, quindi la type safeness può essere garantita, ma la progettazione del linguaggio è più complessa e si possono *rifiutare* programmi che sono sostanzialmente eseguibili, come `if true then "pippo" else 1` che potrebbe venir eseguito senza problemi, oppure dinamici cioè fatti al momento dell'esecuzione. In questo caso un oggetto ha un descrittore a run time che la macchina astratta usa e la macchina astratta controlla i descrittori e prova se sono

corretti. Questo può essere fonte di inefficienza. La soluzione è che spesso si combinano tutti e due i metodi di controllo.

Passiamo in rassegna vari tipi di dato.

- *Booleani*. I *valori* sono vero e falso, le *operazioni* sono le operazioni logiche e la *rappresentazione* dei valori è in generale un byte (per motivi di indirizzabilità).

Il C non ha un tipo booleano.

I booleani sono in generale esprimibili, denotabili e memorizzabili.

- *Caratteri*. I *valori* sono un insieme di codici di caratteri, le *operazioni* sono l'uguaglianza, code/decode, o dipendenti dal linguaggio. La *rappresentazione* è un byte (ASCII), o due byte (UNICODE).

In generale esprimibili, denotabili e memorizzabili.

- *Interi*. I *valori* sono $0, 1, -1, 2, -2, \dots$, maxint, le *operazioni* sono $+, \times, -, \div, mod, \dots$ e la *rappresentazione* è con alcuni byte, e complemento a due.

Interi e interi lunghi (anche 8 byte) possono avere limitati problemi nella portabilità (se la lunghezza non è specificata nella definizione del linguaggio).

In generale esprimibili, denotabili e memorizzabili.

- *Reali*. I *valori* sono i razionali in un certo intervallo, le *operazioni* sono $+, \times, -, \backslash, \dots$ e la *rappresentazione* è data alcuni byte, virgola mobile (standard IEEE 754).

In generale esprimibili, denotabili e memorizzabili.

- *Virgola fissa*. I *valori* sono i razionali in un certo intervallo, le *operazioni* sono $+, \times, -, \backslash, \dots$, e la *rappresentazione* è in generale di alcuni byte, virgola fissa (complemento a 2).

In Ada sono usati per operazioni efficienti con poca precisione.

Sono in generale esprimibili, denotabili e memorizzabili.

- *Complessi*. I *valori* sono un opportuno sottoinsieme di coppie di numeri reali e le *operazioni* sono le solite definite sulle componenti: $+, \times, -, \backslash, \dots$ e ovviamente la loro *rappresentazione* è una coppia di numeri in virgola mobile.

Sono in generale esprimibili, denotabili e memorizzabili.

- *Void*. Il *valore* è un solo elemento, non ci sono operazioni e non ha rappresentazione.

Il valore non è esprimibile, denotabile e memorizzabile

Indica il tipo delle operazioni che modificano lo stato e non restituiscono un valore

Quelli che abbiamo elencato sono tra i *tipi scalari*, detti anche tipi semplici, o di base. Nei tipi scalari i valori non sono aggregazioni di altri valori e hanno una rappresentazione diretta nell'implementazione, inoltre possono essere ordinati (quindi è possibile definire un successore e predecessore), e per questa caratteristica è possibile usarli per iterare, si possono usare come *indici*.

Con il Pascal si introducono i tipi enumerati, seguendo la filosofia che gli elementi costitutivi di un programma siano “algoritmi” e “strutture dati”. Un esempio è il tipo *giorni* dichiarato in questo modo: `type Giorni = (Lun,Mar,Mer,Giov,Ven,Sab,Dom)`, In questo modo si possono scrivere programmi di più semplice comprensione. I valori di un tipo enumerato sono ordinati (ad esempio `Mar < Ven`) ed è quindi possibile compiere iterazione sui valori (possiamo scrivere `for i:= Lun to Sab do ...`), ed hanno un `succ` e `pred`

Sono spesso rappresentati come short integer. In C il tipo `enum giorni = {Lun,Mar,Mer,Giov,Ven,Sab,Dom}` equivale a `typedef int giorni;` e `const giorni Lun=0,Mar=1,Mer=2,...,Dom=6;`, quindi in C `Mar` e il numero 1 sono uguali, mentre nel Pascal sono diversi. In generale i valori dei tipi enumerati sono esprimibili, denotabili e memorizzabili.

Un altro tipo di dato introdotto in Pascal è il tipo “intervallo”, I valori sono un intervallo dei valori di un tipo scalare (il tipo base dell'intervallo), ad esempio `type MenoDiDieci = 0..9` o `type GiorniLav = Lun..Ven`. I valori di questo tipo sono rappresentati come il tipo base. Si usa un tipo intervallo invece del suo tipo base per avere documentazione “controllabile” ed anche generazione di codice efficiente. Sono in generale esprimibili, denotabili e memorizzabili.

I tipi visti fino ad ora sono detti anche *tipi ordinali* e sono i tipi naturali su cui definire gli indici di tipi composti (come gli array).

Tipi composti I tipi composti sono spesso chiamati *tipi strutturati*, o non scalari.

- *Record*: collezione di campi (fields), ciascuno di un (diverso) tipo, ed un campo viene selezionato col suo nome
- *Record varianti*: record dove solo alcuni campi (mutuamente esclusivi) sono attivi ad un dato istante
- *Array*: sono una *funzione* da un tipo indice (scalare) ad un altro tipo. Gli array di caratteri sono chiamati *stringhe*, ed hanno operazioni speciali
- *Insiemi*: sottinsieme di un tipo base (da non confondere con il tipo intervallo)
- *Puntatore*: riferimento (reference) ad un oggetto di un altro tipo

Vediamoli. Il tipo di dato *Record* serve a manipolare in modo unitario dati di tipo eterogeneo (basi di dati). In C, C++, CommonLisp, Algol68 sono definiti con la keyword **struct** (strutture), mentre Java non ha tipi record, sono sussunti dalle classi. Ad esempio, dichiariamo il seguente tipo record (simil C)

```
struct studente {
    char nome[20];
    int matricola;
};
```

e la selezione di campo avviene in questo modo

```
studente s;
s.matricola=343536;
```

I record possono essere annidati, e sono in genere memorizzabili, esprimibili e denotabili. Il Pascal non ha modo di esprimere “un valore record costante” mentre C lo può fare, ma solo nell’inizializzazione (*initializer*), e l’uguaglianza non è generalmente definita (eccezione: Ada). Quando sono memorizzabili, si ha la memorizzazione sequenziale dei campi, con l’allineamento alla parola (16/32 bit), che comporta un potenziale spreco di memoria, ma consente l’accesso con offset. Con i *Packed records* si ha il disallineamento ma accesso più costoso (devo ricordare le posizioni, offset da ricordare).

I *Record varianti* sono record in cui alcuni campi sono alternativi tra loro: solo uno di essi è attivo in un dato istante:

```
type stud = record
    nome : packed array [1..6] of char;
    matricola: integer;
    case fuoricorso : Boolean of
        true: (ultimoanno: 2000..maxint);
        false: (anno:(primo,secondo,terzo);
                inpari: Boolean;)
end;
```

Se avessimo `var s: stud; e s.fuoricorso := true;` allora solo il campo `s.ultimoanno:=2001;` può essere usato. I due campi (le due varianti) `ultimoanno` e `anno` possono condividere la stessa locazione di memoria. Il tipo del tag `fuoricorso` può essere un qualsiasi tipo scalare. I record varianti sono possibili in molti linguaggi. In C abbiamo il tipo `union` e il solito record (`struct`), e l’unione in C rappresenta due alternative mutuamente esclusive

```
struct student {
    char nome[6];
    int matricola;
    bool fuoricorso;
    union {
        int ultimoanno;
        struct {
            int anno;
            bool inpari;} stud_in_corso
    };
};
```

```

        } campivarianti;
    };

```

per accedervi

```

student s;
s.campivarianti.stud_in_corso.inpari

```

Oltre ad una scrittura più complessa che altri linguaggi per l'accesso, il problema è di sicurezza, dato che non c'è un legame tra il valore formale del tag del record e la significatività di una delle varianti (se, ad esempio, contiene un valore e di quale tipo), e anche se introduco controlli non risolvo realmente il problema (il problema sta nell'unione).

```

type Tre = 1..3;
var tmp = record
    case quale : Tre of
        1 : (a : int);
        2 : (b : bool);
        3 : (c : char);
    end

```

e con

```

tmp.quale := 1;
tmp.a := 12;
write(tmp.c) /* errore dinamico

```

ma se prima del `write` faccio `tmp.quale := 3` non c'è errore, anche se il contenuto non è significativo.

Le Unioni in Algol 68 sono invece sicure

```

union (int, bool, char) tmp;
...
tmp := true;
...
tmp := 123;

```

La macchina astratta mantiene per ogni variabile di tipo unione un tag nascosto che viene settato al momento dell'assegnamento, e quando uso la variabile devo prevedere ogni *caso*

```

case tmp of
    (int a) : a := a+1;
    (bool b) : b := false;
    (char c) : print(c);
esac

```

Questo ovviamente rende la programmazione e l'implementazione più pesante.

Gli *array* sono invece collezioni di dati *omogenei* e sono una *funzione* da un tipo indice al tipo degli elementi, quindi il tipo indice è in genere un tipo discreto mentre l'*elemento* può essere un qualsiasi tipo (raramente un tipo funzionale)

Un array lo si dichiara nei seguenti modi:

- C: `int vet[30]`; il tipo indice sono i valori tra 0 e 29
- Pascal: `var vett : array [0..29] of integer;`

Gli array multidimensionali (matrici) sono una funzione da tipo indice a tipo array. In Pascal le seguenti sono equivalenti

```
var mat : array [0..29, 'a'..'z'] of real;  
var mat : array [0..29] of array ['a'..'z'] of real;
```

ma non sono equivalenti in Ada (la seconda permette slicing, vedremo tra poco cosa è). Da ultimo osserviamo che il C fonde array e puntatori. La principale operazione permessa sugli array è la selezione di un elemento (`vet[3]` o `mat[10, 'c']`), mentre la modifica non è un'operazione sull'array, ma sulla locazione modificabile che memorizza l'elemento di un array. Alcuni linguaggi permettono lo slicing, cioè la selezione di parti contigue di un array (in Ada, data la dichiarazione `mat : array(1..10) of array(1..10) of real;`, con `mat(3)` si indica la terza riga della matrice quadrata `mat`). Se si ha lo slicing si possono avere anche operazioni sugli array:

- Fortran 90: `A+B` somma gli elementi di A e B (dello stesso tipo)
- l'APL è linguaggio (dei primissimi anni '60) per manipolazioni di array, e ha potenti operazioni su array, è un linguaggio sintetico e austero, il lessico è *matematico* (lettere greche) ed è impossibile da leggere (e da scrivere sulle telescriventi dell'epoca)

La notazione è importante: *Notation as a tool for thought* di Kenneth E. Iverson, ACM Turing Lecture del 1979

La memorizzazione degli array è stata molto importante (in generale gli array vanno nel RdA) Gli elementi sono in genere memorizzati in locazioni contigue, e l'ordine può essere o di riga o di colonna. Ordine di riga: `V[1,1];V[1,2];...;V[1,10];V[2,1];...`, è quello maggiormente usato, il "subarray" di un array di array (ad esempio la terza riga) sta in locazioni contigue. Nell'ordine di colonna `V[1,1];V[2,1];V[3,1];...;V[10,1];V[1,2];...` le colonne stanno in locazioni contigue. L'ordine è rilevante per l'efficienza in sistemi con cache. Come calcolare degli indirizzi: per calcolare la locazione corrispondente a `A[i,j,k]` (memorizzato per riga)

```
A : array[l1..u1, l2..u2, l3..u3] of elem_type;
```

- S3 dimensione di (un elemento di) `elem_type`
- $S2 = (u3 - l3 + 1) * S3$ è la dimensione di una riga

- $S1 = (u2-12+1)*S2$ è la dimensione di un piano

la locazione di $A[i, j, k]$ è

α indirizzo di inizio di A
 $+ (i-11)*S1$ indirizzo di inizio del piano di $A[i, j, k]$
 $+ (j-12)*S2$ indirizzo di inizio della riga di $A[i, j, k]$
 $+ (k-13)*S3$ indirizzo di $A[i, j, k]$

Se il numero di dimensioni è noto a tempo di compilazione, si può riorganizzare come segue $i*S1+j*S2+k*S3+ \alpha - (11*S1+12*S2+13*S3)$, e se l'indice inizia sempre da zero ($11=12=13=0$) non c'è bisogno di precomputazione.

La *forma*, o *shape*, di un array è il numero delle dimensioni e l'intervallo dell'indice. Se la forma è statica al momento della compilazione (nell'RdA si prevede lo spazio che serve) oppure lo si fa al momento dell'elaborazione della dichiarazione, e anche in questo caso nell'RdA si prevede lo spazio che serve, lo si fa nel codice del preambolo, ma l'RdA si divide nella parte dati di dimensioni *fisse* e quella di dati di dimensioni *non determinabili* al momento della compilazione. Le informazioni su come trovare effettivamente gli elementi devono essere mantenute in qualche modo (il *dope vector*). Nel caso la forma sia dinamica, allora l'array può cambiare forma durante l'esecuzione del programma.

Il *dope vector* mantiene le informazioni sulla forma dell'array, che nel caso statico non può essere manipolata se non al momento della dichiarazione. Il *dope vector* di un array contiene:

- il puntatore all'inizio dell'array (nella parte variabile dell'RdA)
- il numero delle dimensioni
- il limite inferiore (se la forma cambia a run-time vogliamo controllare anche l'out-of-bound, quindi serve anche il limite superiore)
- occupazione di memoria per ogni dimensione (valore S_i)

Il *dope vector* è memorizzato nella parte fissa del RdA. L'accesso viene effettuato calcolando tutto l'indirizzo a run-time, usando il *dope vector*.

L'array in Java non c'è. `int[] V;` equivale a fare `V = new int[]` che alloca spazio sullo heap e dà il suo riferimento a `V`.

Vediamo ora i *Puntatori*. Dato un *indirizzo* (locazione), *dereferenziare* significa accedere al dato *puntato* (indirizzato) dall'indirizzo. Nel seguito locazione e *l*-valore saranno sinonimi, e i puntatori sono le locazioni. Ci sono linguaggi con variabili modificabili che permettono di riferirsi (e modificare) un *l*-valore senza dereferenziarlo, promuovendo gli *l*-valori a dati che si manipolano. Quindi abbiamo che per i puntatori abbiamo che i *valori* sono proprio le locazioni (*l*-valori o riferimenti) e `null` (o `nil`) è il puntatore che non punta a nulla. Le *operazioni* sono la creazione, la dereferenziazione e l'uguaglianza.

La loro *rappresentazione* è data dagli indirizzi. Se gli oggetti puntati sono di un certo tipo (T), i puntatori hanno in qualche modo anche il tipo dell'oggetto puntato (si indica spesso con T*), e in alcuni linguaggi i puntatori possono essere trattati aritmeticamente (ma si perde certamente la type safeness).

In C i puntatori possono essere creati a partire dal dato

```
float r = 3.1415;
float* q;
q = &r
```

in q c'è l'indirizzo di r.

I puntatori sono in generale usati per le strutture dati ricorsive

```
type int_list = elemento*;
type elemento = struct {
    int val;
    int_list next;
};
```

In C array e puntatori sono la stessa cosa

```
int n;
int *a;      // puntatore a interi
int b[10];   // array di 10 interi
...
a = b;       // a punta all'elemento iniziale di b
n = a[3];    // n ha il valore del terzo elemento di b
n = *(a+3);  // idem
n = b[3];    // idem
n = *(b+3);  // idem
```

array multidimensionali: `b[i][j]` è la stessa cosa di `*(b+i)[j]` o `*(b[i]+j)` o ancora `*(b+i+j)`.

Un puntatore implica in generale che si alloca della memoria sullo heap, e questa si esaurisce, ma le locazioni libere vengono recuperate con la *garbage collection*. Si può prevedere la deallocazione esplicita, ma il problema è il solito: la vita breve e i dangling reference.

```
int* p;
int* q;
p = (int*) malloc(sizeof(int));
*p = 5;
q = p;    // alias
free(p);
p = null;
```

```
...          // non modifico p o q
print(*p)    // la macchina astratta rileva l'errore (grazie al null)
print(*q)    // errore non rilevabile: dangling reference...
```

I *tipi ricorsivi* sono tipi composti da valori di un tipo e riferimenti a valori di quel tipo, gli oggetti di questo tipo sono allocati sullo heap e dipendono fortemente dal linguaggio. Nel ML i tipi ricorsivi sono presenti esplicitamente e non devo preoccuparmi di allocazioni (una buona norma nella definizione di un tipo ricorsivo è la stessa per la definizione induttiva: bisogna avere i casi base), mentre in Java l'allocazione avviene con la **new** (che serve per allocare lo spazio sullo heap) e c'è anche il **null**.

Da ultimo vediamo il tipo *funzione*. In generale le funzioni sono denotabili e, in linguaggi funzionali, esprimibili, ma in generale non sono memorizzabili.

Sistema di tipi, equivalenza, compatibilità, polimorfismo e regole

Prima di vedere cosa è un sistema di tipi effettivamente ricordiamo che abbiamo visto come formare tipi a partire da altri tipi. Possiamo ora domandarci quali sono le regole che governano la correttezza di un programma rispetto ai tipi. I nuovi tipi li dichiariamo in questo modo

```
type nome = espressione
```

dove **espressione** è ottenuta combinando opportunamente i costruttori e tipi base visti prima. Abbiamo due tipi di definizione di tipo, una è la definizione detta *opaca* (che ha per equivalenza quella per *nome*) e l'altra è la definizione *trasparente* (che ha per equivalenza quella *strutturale*).

L'equivalenza è una relazione transitiva, riflessiva e simmetrica, mentre la compatibilità è una relazione transitiva e riflessiva.

Definizione 22 *Due tipi sono equivalenti per nome solo se hanno lo stesso nome.*

Un tipo è equivalente solo a se stesso (usata in Pascal, Ada, Java). In Pascal questi sono due tipi diversi (non hanno lo stesso nome):

```
var V : array[1..10] of integer;
    W : array[1..10] of integer;
```

L'equivalenza può essere *lasca* (loose, in Pascal o Modula-2): una dichiarazione di un alias di tipo non genera un nuovo tipo, ma solo un nuovo nome

```
type A : espressione;
type B = A;
```

A e B sono due nomi dello stesso tipo.

Definizione 23 *Si consideri la minima relazione d'equivalenza sui tipi (indicata con $\sim$) tale che*

- un nome di tipo è equivalente a se stesso
- se un tipo T è introdotto con una definizione di tipo *type* $T = \text{espressione}$, allora T è equivalente a *espressione*
- se due tipi T e S sono costruiti usando lo stesso costruttore di tipo a tipi equivalenti, allora sono equivalenti

Due tipi T e S sono equivalenti strutturalmente se solo se $T \sim S$

Se abbiamo

```
type T1 = int;
type T2 = char;
type T3 = struct {
    T1 a;
    T2 b;}
type T4 = struct {
    int a;
    char b;}
```

allora $T3 \sim T4$.

L'equivalenza strutturale è governata dalla riscrittura: un tipo complesso viene *riscritto* nei suoi componenti elementari. A basso livello non rispetta l'astrazione desiderata dal programmatore. Quasi tutti i linguaggi hanno un qualche tipo d'equivalenza (o strutturale, o per nome, o per nome loose).

Definizione 24 T è compatibile con S quando oggetti di T possono essere usati in un contesto dove ci si attende valori S .

La definizione dipende in modo cruciale dal linguaggio. T è compatibile con S se

- T e S sono equivalenti
- i valori di T sono un sotto insieme dei valori di S (un intervallo)
- tutte le operazioni sui valori di S sono possibili anche sui valori di T (principio di *estensione* di record)
- i valori di T corrispondono in modo canonico ad alcuni valori di S (ad esempio `int` e `float`)
- i valori di T possono essere fatti corrispondere ad alcuni valori di S (`float` e `int` con troncamento)

Tra tipi compatibili si può anche definire (implicitamente o esplicitamente) un'operazione di conversione. implicita (coercion) o esplicita (indicata nel testo)

La *coercion* è una conversione implicita (detta anche coercizione): la macchina astratta inserisce la conversione, senza che ve ne sia traccia a livello linguistico. La coercizione serve per indicare una situazione di compatibilità e per indicare cosa deve fare l'implementazione.

Se i tipi sono diversi ma hanno gli stessi valori e stessa rappresentazione (ad esempio, tipi strutturalmente uguali ma nomi diversi), la conversione c'è solo a tempo di compilazione, non c'è codice; se invece i tipi hanno valori diversi, ma la stessa rappresentazione nell'intersezione (ad esempio intervalli e interi), allora c'è del codice per il controllo dinamico sull'appartenenza all'intersezione, infine se hanno valori e rappresentazione diversi (ad esempio, interi e reali), allora c'è bisogno di codice per la conversione.

Il *cast* è invece un operatore di conversione esplicita: il programmatore deve inserire esplicite conversioni di tipo. Possiamo considerarle come annotazioni nel linguaggio che specificano che un valore di un tipo deve essere convertito in un altro tipo. Il **cast** è un costrutto del linguaggio. Non è consentita ogni conversione esplicita, ma solo quelle per le quali il linguaggio conosce come implementare la conversione, e si può sempre inserire un cast laddove esiste una compatibilità. I linguaggi moderni tendono a favorire il cast rispetto alla coercion.

Polimorfismo

Definizione 25 *Un sistema di tipi nel quale uno stesso oggetto può avere più di un tipo è detto polimorfo. Un oggetto è polimorfo quando il sistema di tipi gli assegna più di un tipo.*

Si può avere polimorfismo *ad hoc* (overloading), o polimorfismo *universale* (esplicito, implicito, di sottotipo). L'*overloading* non è vero polimorfismo. Uno stesso simbolo denota significati diversi, ad esempio se abbiamo '3 + 5' e '4.5 + 5.3' il compilatore traduce il + in modi diversi, a seconda del contesto. L'*overloading* è sempre risolto a tempo di compilazione, dopo l'inferenza dei tipi, e non va confuso con la coercion. Anche se spesso hanno lo stesso effetto, rispondono a problemi diversi.

Definizione 26 *Un valore esibisce polimorfismo universale parametrico quando ha un'infinità di tipi diversi, che si ottengono per istanziazione da un unico schema di tipo generale.*

Una funzione polimorfa è costituita da un unico codice che si applica uniformemente a tutte le istanze del suo tipo generale

```
void swap (reference <T> x, reference <T> y) {  
<T> tmp = x;  
x := y;  
y := tmp;  
}
```

```

x := y;
y := z;
}

```

`swap` ha tipo $\langle T \rangle \times \langle T \rangle \rightarrow \text{void}$ che viene indicato anche con l'espressione di tipo $\forall T. \langle T \rangle \times \langle T \rangle \rightarrow \text{void}$

Il polimorfismo qui è esplicito, dato che si indica lo schema di tipo. Può essere implicito: il programma non riporta indicazioni sul tipo, ma queste vengono ottenute tramite inferenza (ad esempio *ML*).

Definizione 27 *Un valore esibisce polimorfismo di sottotipo (o limitato) quando ha un'infinità di tipi diversi, che si ottengono per istanziazione da uno schema di tipo generale, sostituendo ad un opportuno parametro i sottotipi di un tipo assegnato.*

Il polimorfismo di *sottotipo* è analogo a quello esplicito, ma non tutti i tipi possono essere usati per istanziare il tipo generale, c'è una relazione (data) di sottotipo. $T < S$ significa che T è sottotipo di S .

Una funzione polimorfa è costituita da un unico codice che si applica uniformemente a tutte le istanze *legali* del suo tipo generale, ossia $\forall T < D. \langle T \rangle [] \rightarrow \text{void}$. Il polimorfismo di *sottotipo* è tipico del paradigma object oriented.

Sicurezza (rispetto ai tipi)

I linguaggi *non sicuri* sono quelli in cui il sistema di tipi è un suggerimento metodologico per il programmatore, ma il linguaggio permette di aggirare o rilassare i controlli (esempio C, C++ e linguaggi della famiglia), i linguaggi *localmente non sicuri* sono quelli in cui il sistema di tipi è ben regolato e i tipi controllati ma ci sono alcuni costrutti che consentono di aggirare i controlli (Algol, Pascal, Ada), e la locale non sicurezza dipende principalmente dalle unioni (varianti non controllate) e deallocazione esplicita. I linguaggi *sicuri* sono quelli in cui il sistema di tipi è ben regolato e i tipi controllati e non ci sono costrutti per aggirare i controlli (Java, ML, Lisp, Scheme).

Vediamo ora le regole per i tipi. In questo caso i tipi sono definiti da una grammatica, ad esempio

$$\tau ::= \text{int} \mid \tau_1 * \tau_2 \mid \tau_1 \Rightarrow \tau_2$$

abbiamo bisogno di un linguaggio

$$t ::= x \mid n \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid (t_1, t_2) \mid \text{fst}(t) \mid \text{snd}(t) \mid \\ \mid \lambda x. t \mid \text{apply}(t_1, t_2) \mid \text{let } x \leftarrow t_1 \text{ in } t_2 \mid \text{rec } y. (\lambda x. t)$$

ed abbiamo un sistema di regole per assegnare ad ogni espressione del linguaggio un tipo:

$$\begin{array}{c}
\frac{}{x : \text{type}(x)} \qquad \frac{}{n : \mathbf{int}} \qquad \frac{t_1 : \mathbf{int} \quad t_2 : \mathbf{int}}{t_1 \text{ op } t_2 : \mathbf{int}} \text{ op} \in \{+, -, \times\} \\
\\
\frac{t_0 : \mathbf{int} \quad t_1 : \tau \quad t_2 : \tau}{\text{if } t_0 \text{ then } t_1 \text{ else } t_2 : \tau} \quad \frac{t_1 : \tau_1 \quad t_2 : \tau_2}{(t_1, t_2) : \tau_1 * \tau_2} \quad \frac{t : \tau_1 * \tau_2}{fst(t) : \tau_1} \\
\\
\frac{t : \tau_1 * \tau_2}{snd(t) : \tau_2} \quad \frac{x : \tau_1 \quad t : \tau_2}{\lambda x. t : \tau_1 \Rightarrow \tau_2} \quad \frac{t_1 : \tau_1 \Rightarrow \tau_2 \quad t_2 : \tau_1}{\text{apply}(t_1, t_2) : \tau_2} \\
\\
\frac{x : \tau_1 \quad t_1 : \tau_1 \quad t_2 : \tau_2}{\text{let } x \Leftarrow t_1 \text{ in } t_2 : \tau_2} \quad \frac{y : \tau \quad \lambda x. t : \tau}{\text{rec } y. (\lambda x. t) : \tau}
\end{array}$$

Inferenza di tipo

Considerate il linguaggio di programmazione, il cui scope è statico (il linguaggio riflette lo scope statico quando presenta esplicitamente un costrutto per gestire la ricorsione), e la cui sintassi è la seguente:

$$\begin{aligned}
t ::= & \ x \mid n \mid c \mid \text{true} \mid \text{false} \mid \text{empty} \mid t_1 + t_2 \mid t_1 - t_2 \mid t_1 \times t_2 \mid t_1 \wedge t_2 \mid t_1 \vee t_2 \mid \neg t_1 \mid t_1 = t_2 \\
& \mid t_1 < t_2 \mid \text{cons}(t_1, t_2) \mid \text{head}(t) \mid \text{tail}(t) \mid \text{fst}(t) \mid \text{snd}(t) \mid \text{pair}(t_1, t_2) \mid \\
& \mid \text{if } t_0 \text{ then } t_1 \text{ else } t_2 \mid \text{fun}(x, t) \mid \text{apply}(t_1, t_2) \mid \text{let } x \Leftarrow t_1 \text{ in } t_2 \mid \text{rec } y. (\text{fun}(x, t))
\end{aligned}$$

dove n sono interi e c caratteri, *empty* indica la lista vuota, *cons*, *head* e *tail* sono le ovvie operazioni sulle liste (costruzione, selezione del primo elemento e del resto di una lista), *pair* è il costruttore delle coppie e avete anche gli ovvi selettori (*fst* e *snd*), e le altre parti sono standard. Il predicato di uguaglianza è polimorfo, quindi lo potete usare in qualsiasi contesto abbia senso usarlo.

Considerate inoltre la seguente grammatica per i tipi

$$\tau ::= a \mid \mathbf{int} \mid \mathbf{bool} \mid \mathbf{char} \mid \tau \text{ list} \mid \tau_1 * \tau_2 \mid \tau_1 \rightarrow \tau_2$$

dove $a \in A$ sono le variabili di tipo e le seguenti regole descrivono come si fa ad attribuire un tipo ad un termine.

$$\begin{array}{c}
\frac{}{x : \text{type}(x)} \quad \frac{}{n : \text{int}} \quad \frac{}{c : \text{char}} \quad \frac{t_1 : \text{int} \quad t_2 : \text{int}}{t_1 \text{ op } t_2 : \text{int}} \text{ op} \in \{+, -, \times\} \\
\\
\frac{}{\text{true} : \text{bool}} \quad \frac{}{\text{false} : \text{bool}} \quad \frac{t_1 : \text{bool} \quad t_2 : \text{bool}}{t_1 \text{ op } t_2 : \text{bool}} \text{ op} \in \{\wedge, \vee\} \quad \frac{t_1 : \tau \quad t_2 : \tau}{t_1 = t_2 : \text{bool}} \\
\\
\frac{t : \text{bool}}{\neg t : \text{bool}} \quad \frac{t_1 : \text{int} \quad t_2 : \text{int}}{t_1 < t_2 : \text{bool}} \quad \frac{t_1 : \tau_1 \quad t_2 : \tau_2}{\text{pair}(t_1, t_2) : \tau_1 * \tau_2} \\
\\
\frac{t : \tau_1 * \tau_2}{\text{fst}(t) : \tau_1} \quad \frac{t : \tau_1 * \tau_2}{\text{snd}(t) : \tau_2} \quad \frac{}{\text{empty} : \tau \text{ list}} \quad \frac{t_1 : \tau \quad t_2 : \tau \text{ list}}{\text{cons}(t_1, t_2) : \tau \text{ list}} \quad \frac{t : \tau \text{ list}}{\text{head}(t) : \tau} \\
\\
\frac{t : \tau \text{ list}}{\text{tail}(t) : \tau \text{ list}} \quad \frac{t_0 : \text{bool} \quad t_1 : \tau \quad t_2 : \tau}{\text{if } t_0 \text{ then } t_1 \text{ else } t_2 : \tau} \quad \frac{x : \tau_1 \quad t : \tau_2}{\text{fun}(x, t) : \tau_1 \rightarrow \tau_2} \\
\\
\frac{t_1 : \tau_1 \rightarrow \tau_2 \quad t_2 : \tau_1}{\text{apply}(t_1, t_2) : \tau_2} \quad \frac{x : \tau_1 \quad t_1 : \tau_1 \quad t_2 : \tau_2}{\text{let } x \leftarrow t_1 \text{ in } t_2 : \tau_2} \quad \frac{y : \tau \quad \text{fun}(x, t) : \tau}{\text{rec } y.(\text{fun}(x, t)) : \tau}
\end{array}$$

Assumiamo sempre che i termini che dobbiamo valutare siano *ben tipati*, nel senso che riusciamo sempre ad attribuire loro un tipo (che può contenere variabili di tipo).

L'algoritmo di inferenza di tipo usa le regole per l'attribuzione del tipo elencate prima per costruire un insieme di vincoli che poi dovrà essere risolto. I vincoli vengono costruiti nel seguente modo. Dato un termine t e la funzione type che associa ad ogni variabile del linguaggio un tipo τ della grammatica dei tipi, generiamo una coppia formata dal tipo di t e da un insieme di vincoli, quindi riscriviamo le regole in modo più operativo. La funzione $\text{newvar}()$ genera una nuova variabile di tipo.

$$\begin{array}{c}
\frac{}{\langle\langle x, \text{type} \rangle\rangle \Rightarrow (\text{type}(x), \emptyset)} \quad \frac{}{\langle\langle n, \text{type} \rangle\rangle \Rightarrow (\text{int}, \emptyset)} \quad \frac{}{\langle\langle c, \text{type} \rangle\rangle \Rightarrow (\text{char}, \emptyset)} \\
\\
\frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 \text{ op } t_2, \text{type} \rangle\rangle \Rightarrow (\text{int}, \{(\tau_1, \text{int}), (\tau_2, \text{int})\} \cup T_1 \cup T_2)} \text{ op} \in \{+, -, \times\} \quad \frac{}{\langle\langle \text{true}, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \emptyset)} \\
\\
\frac{\langle\langle t_1, \text{type} \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, \text{type} \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 \text{ op } t_2, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \{(\tau_1, \text{bool}), (\tau_2, \text{bool})\} \cup T_1 \cup T_2)} \text{ op} \in \{\wedge, \vee\} \quad \frac{}{\langle\langle \text{false}, \text{type} \rangle\rangle \Rightarrow (\text{bool}, \emptyset)}
\end{array}$$

$$\begin{array}{c}
\frac{\langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 < t_2, type \rangle\rangle \Rightarrow (\text{bool}, \{(\tau_1, \text{int}), (\tau_2, \text{int})\} \cup T_1 \cup T_2)} \quad \frac{\langle\langle t, type \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \neg t, type \rangle\rangle \Rightarrow (\text{bool}, \{(\tau, \text{bool})\} \cup T)} \\
\\
\frac{\langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle t_1 = t_2, type \rangle\rangle \Rightarrow (\text{bool}, \{(\tau_1, \tau_2)\} \cup T_1 \cup T_2)} \quad \frac{\langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle pair(t_1, t_2), type \rangle\rangle \Rightarrow (\tau, \{(\tau, \tau_1 * \tau_2)\} \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t, type \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle fst(t), type \rangle\rangle \Rightarrow (\tau_1, \{(\tau, \tau_1 * a)\} \cup T)} \quad \frac{a = \text{newvar}() \quad \langle\langle t, type \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle snd(t), type \rangle\rangle \Rightarrow (\tau_2, \{(\tau, a * \tau_2)\} \cup T)} \\
\\
\frac{a = \text{newvar}()}{\langle\langle empty, type \rangle\rangle \Rightarrow (a \text{ list}, \emptyset)} \quad \frac{\langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle cons(t_1, t_2), type \rangle\rangle \Rightarrow (\tau_1 \text{ list}, \{((\tau_2, \tau_1 \text{ list}))\} \cup T_1 \cup T_2)} \\
\\
\frac{\langle\langle t, type \rangle\rangle \Rightarrow (\tau_2, T)}{\langle\langle head(t), type \rangle\rangle \Rightarrow (\tau, \{(\tau_2, \tau \text{ list})\} \cup T)} \quad \frac{\langle\langle t, type \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle tail(t), type \rangle\rangle \Rightarrow (\tau, \{(\tau, \tau_1 \text{ list})\} \cup T)} \\
\\
\frac{\langle\langle t_0, type \rangle\rangle \Rightarrow (\tau_0, T_0) \quad \langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle \text{if } t_0 \text{ then } t_1 \text{ else } t_2, type \rangle\rangle \Rightarrow (\tau, \{(\tau_0, \text{bool}), (\tau, \tau_1), (\tau_1, \tau_2)\} \cup T_0 \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t, type[x/a] \rangle\rangle \Rightarrow (\tau_2, T)}{\langle\langle fun(x, t), type \rangle\rangle \Rightarrow (a \rightarrow \tau_2, T)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t_1, type \rangle\rangle \Rightarrow (\tau, T_2) \quad \langle\langle t_2, type \rangle\rangle \Rightarrow (\tau_1, T_1)}{\langle\langle apply(t_1, t_2), type \rangle\rangle \Rightarrow (a, \{(\tau, \tau_1 \rightarrow a)\} \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle t_1, type \rangle\rangle \Rightarrow (\tau_1, T_1) \quad \langle\langle t_2, type[x/a] \rangle\rangle \Rightarrow (\tau_2, T_2)}{\langle\langle \text{let } x \leftarrow t_1 \text{ in } t_2, type \rangle\rangle \Rightarrow (\tau_2, \{(\tau_1, a)\} \cup T_1 \cup T_2)} \\
\\
\frac{a = \text{newvar}() \quad \langle\langle fun(x, t), type[y/a] \rangle\rangle \Rightarrow (\tau, T)}{\langle\langle \text{rec } y.(fun(x, t)), type \rangle\rangle \Rightarrow (\tau, \{(\tau, a)\} \cup T)}
\end{array}$$

Una volta ottenuto l'insieme di vincoli, questi vanno *risolti* tramite un algoritmo che si basa sulla *sostituzione* e sull'*unificazione*. Una sostituzione è una funzione che associa ad ogni variabile di tipo un tipo (con eventuali variabili). Ovviamente l'identità è una sostituzione (ogni variabile di tipo viene *mandata* in se stessa) e le sostituzioni possono venire *composte*. Dato un vincolo, che è una coppia, si cerca la sostituzione che rende i due elementi della coppia uguali. Quindi, dato un insieme C di coppie, per trovare la sostituzione corretta si procede come segue, partendo dalla sostituzione *identica*. La sostituzione la chiamiamo s .

- se la coppia è fatta da due termini uguali, la si elimina
- se la coppia è fatta da una variabile a ed un termine q in cui tale variabile non occorre allora si aggiorna la sostituzione s ponendo che $s(a) = q$ e si applica tale sostituzione a tutte le coppie in C

- se la coppia è fatta da due termini composti da costruttori (nel nostro caso coppie, liste e frecce) allora si verifica che i costruttori più esterni siano uguali, e se lo sono si aggiungono le coppie fatte dai componenti accoppiati secondo le regole dell'equivalenza e si elimina la coppia di partenza
- si ripetono i passi fino a che non si ottiene che C è vuoto (ed allora abbiamo la sostituzione cercata) oppure se si incontra una coppia che non è unificabile

Per chiarire, alcuni esempi: $\{(int, int)\}$ unifica con la sostituzione identica, $\{(a, int)\}$ unifica con la sostituzione $s(a) = int$, $\{(int, a \rightarrow b)\}$ non unifica mentre $\{(a \rightarrow int, bool \rightarrow b)\}$ unifica con la sostituzione $s(a) = bool$ e $s(b) = int$. Osservate che da $\{(a \rightarrow int, bool \rightarrow b)\}$ si sono generati $\{(a, bool), (int, b)\}$.

Astrarre sui dati

Come nel caso dell'astrazione sul controllo o funzionale, si può realizzare anche l'astrazione sui dati. Le ragioni sono simili. La prima è per nascondere dettagli non necessari, e quindi focalizzarsi su quelli che interessano davvero, e la seconda è l'aumento della portabilità.

Come si realizza l'astrazione sui dati? Abbiamo i seguenti concetti:

- *componente*: unità di programma (ad esempio funzione, struttura dati, modulo)
- *interfaccia*: tipi e operazioni definiti in un componente che sono visibili fuori del componente stesso
- *specifica*: è il funzionamento *inteso* del componente, espresso mediante proprietà osservabili attraverso l'interfaccia
- *implementazione*: strutture dati e funzioni definiti dentro al componente, non necessariamente visibili da fuori

L'idea è di formalizzare il comportamento inteso senza parlare della sua implementazione, e poi realizzare un'implementazione che però non è visibile all'utente (*information hiding*).

Vediamo un esempio. Per la pila devono valere le seguenti proprietà (ci sono tre operazioni ed un predicato, le solite, la pila vuota la indichiamo con `Empty`):

1. `pop(push(e,p)) = p`
2. `top(push(e,p)) = e`
3. `push(top(p),pop(p)) = p`
4. `isempty(Empty) = true` e `isempty(push(e,p)) = false`

Ogni implementazione deve essere tale che la specifica sia soddisfatta.

Un tipo di dato è in generale *fatto* da un insieme di valori base e di operazioni, nell'astrazione nascondiamo come rappresentiamo gli elementi del tipo (ad esempio come realizziamo davvero la pila), implementiamo le operazioni in modo invisibile all'utente e la specifica dice come vengono manipolati i dati. L'implementazione è *inaccessibile* all'utente, perché protetta da una capsula che la isola.

Se l'astrazione sul controllo significa *nascondere* la realizzazione dei corpi di procedure/funzioni, astrarre sui dati significa nascondere decisioni sulla rappresentazione delle strutture dati e nascondere l'implementazione delle operazioni.

Il supporto linguistico fornito da un linguaggio per l'*information hiding* è la nozione di *Abstract Data Type* (ADT) che è tra i maggiori contributi ai linguaggi di programmazione degli anni '70. L'idea è semplice: separare l'interfaccia (quali operazioni e simili) dall'implementazione. **Sets** è un tipo di dato in cui ho operazioni come **empty**, **insert**, **union**, **ismember**. La specifica di queste operazioni è chiara, e gli insiemi li posso implementare come voglio (liste, vettori, ecc.). Si usa il controllo di tipo per garantire la separazione: il cliente ha accesso alle sole operazioni dell'interfaccia, mentre l'implementazione vera e propria è *incapsulata* in opportuni costrutti opachi.

In un ADT ci sono in generale:

- *costruttori*: operazioni per costruire gli elementi del tipo (usando eventualmente dati di altri tipi): **creapila**, che crea una pila vuota,
- *trasformatori* (o operatori): operazioni che calcolano valori di quel tipo ed eventualmente lo restituiscono: **push** oppure **pop**,
- *osservatori*: operazioni che restituiscono valori di altri tipi basati su proprietà dell'elemento: **top**.

Con gli ADT c'è un importante principio: indipendenza dalla rappresentazione. Infatti *due implementazioni corrette di un tipo (astratto) non sono distinguibili dai clienti di quel tipo*, quindi le implementazioni sono modificabili senza con ciò interferire con alcun cliente, e il cliente non ha alcun modo per accedere all'implementazione

Il *modulo* è il costrutto generale per l'*information hiding*, ed è disponibile sia in linguaggi imperativi e funzionali che, ovviamente, in quelli object oriented. Il modulo ha due parti:

- *interfaccia*: un insieme di nomi e relativi tipi
- *implementazione*: dichiarazioni (di tipi e funzioni) per ogni nome dell'interfaccia ed eventuali dichiarazioni aggiuntive (nascoste al cliente)

I costrutti sono, per esempio, i moduli di **Modula**, packages di **Ada**, strutture di **ML**.

Gli ADT hanno un importante limite: sono *rigidi* (difficilmente estendibili), dato che le aggiunte modificano l'ADT senza alcuna relazione codificate con l'ADT di partenza.

Invece c'è bisogno di costrutti che permettano la definizione di nuovi tipi e operazioni in modo astratto, che rispettino i principi d'incapsulamento (dell'*information hiding*). Quindi di costrutti per

- *information hiding* e incapsulamento
- riuso del codice (ereditarietà)
- compatibilità tra tipi (sottotipi)
- selezione dinamica delle operazioni

La soluzione proposta è quella di *oggetto*. Un oggetto è una *capsula* che contiene

- dati nascosti: variabili, valori (talvolta anche operazioni)
- operazioni pubbliche: operazioni, dette metodi

In un programma orientato agli oggetti si mandano *messaggi* agli oggetti (il messaggio è l'invocazione del metodo di quell'oggetto), e l'oggetto risponde ai messaggi (lo stato è confinato negli oggetti). Principi di organizzazione permettono di raggruppare gli oggetti con la stessa struttura (classi) e questi stessi principi consentono estensibilità e riuso. Va sottolineato che l'*astrazione* sui dati e sul controllo, l'*information hiding* e l'*incapsulamento* sono presenti nel progetto del linguaggio di programmazione sin dall'inizio.

Definizione 28 *Una classe è un modello di un insieme di oggetti, quali dati, nome, segnatore e visibilità delle operazioni (metodi).*

Gli oggetti sono creati dinamicamente per istanziazione di una classe, mediante costruttori

```
class Counter {
    private int x;
    public void reset () {
        x = 0
    }
    public int get () {
        return x;
    }
    public void inc () {
        x = x + 1;
    }
}
```

Le classi garantiscono l'incapsulamento (opportuni modificatori assicurano che determinati campi siano *pubblici*, *privati* o *protetti*), la parte pubblica è l'*interfaccia* della classe, mentre la parte privata è l'implementazione. Altri meccanismi assicurano che l'incapsulamento sia compatibile con l'estensibilità e la modificabilità del codice e con i modi per il riuso (**extending**) e ridefinizione (**overriding**) di metodi.

Ad esempio, **NewCounter** estende l'interfaccia di **Counter** con nuovi campi e ridefinisce il metodo **reset** (e il messaggio **reset** inviato ad un'istanza di **NewCounter** causa l'invocazione del nuovo codice...):

```
class NamedCounter extending Counter {
    private string Name;
    public SetName (String n) {
        Name = n;
    }
}
```

Con la keyword **extending** si creano nuovi tipi (sottotipi di uno più generale). Un tipo può avere più di un sovratipo immediato (ad esempio, in **Java** quando i sovratipi sono classi totalmente astratte, senza implementazioni, che nel gergo **Java** sono chiamate implementazioni), per cui la gerarchia dei sottotipi non è un albero. Il *sottotipo* fornisce la possibilità di usare un oggetto in un altro contesto (sostanzialmente è una relazione tra le *interfacce* di due classi), mentre l'*ereditarietà* dà la possibilità di riusare il codice che manipola un oggetto (quindi è una relazione tra le *implementazioni* di due classi).

L'ereditarietà permette il riuso del codice in un contesto estendibile (ogni modifica all'implementazione di un metodo in una classe è automaticamente disponibile in tutte le sottoclassi): **NewCounter** eredita da **Counter** i metodi (e i campi) non ridefiniti.

Le nozioni di sottotipo ed ereditarietà forniscono due meccanismi che sono logicamente del tutto indipendenti ma (spesso) linguisticamente collegati nei linguaggi object oriented. Ad esempio, sia **C++** che **Java** hanno costrutti che introducono contemporaneamente entrambe le relazioni tra due classi: **extends** in **Java** introduce un sottotipo e può introdurre ereditarietà se ci sono metodi della superclasse che non sono ridefiniti nella sottoclasse. In **Java** l'ereditarietà è singola: ogni classe eredita al più da una sola superclasse immediata, mentre in **C++** l'ereditarietà può essere multipla: una classe può ereditare da più di una superclasse immediata.